InfraNet Pro
NETWORK DOCUMENTATION

TECHNICAL MANUAL

Document your network the way you see it.

Visual mapping of your network infrastructure: floor plan, 19″ rack, L1/L2 topology, SNMP discovery, VLAN management and handover dossier — in a single application, installed on your own premises.

30
DEVICE TYPES

v1·v2c·v3
NATIVE SNMP DISCOVERY

16
THEMATIC CHAPTERS

On-premise · self-hosted | **Manual-first** | Documentation check (drift) | Label printing · **Avery / Dymo** | Bilingual IT / EN

Complete technical manual · One volume
Every feature of the app

Version: 2.9.2 · August 2026
Real app colours · A4-printable

Contents

A clear guide to InfraNet Pro's features — written for network engineers, system integrators and MSPs.

01 Requirements & installation

What you need to run InfraNet Pro and how to get it up and running in a few minutes.

InfraNet Pro is an **on-premise** application: it runs entirely on your own server or computer, with no cloud services, no external databases and no mandatory containers. Your data stays in-house.

What you need

Component	Requirement
System	Windows, Linux or macOS — any computer/server that stays on
Runtime	**Node.js 16 or later** (LTS recommended)
Browser	A modern browser: up-to-date Chrome, Edge or Firefox
Network	Reachability to the devices you want to document via **SNMP** (UDP 161) — only needed if you use automatic discovery
Disk space	Minimal: projects are lightweight files saved locally

Where to get Node.js

Download it for free from the official site **nodejs.org**: choose the **LTS** version (the most stable) for your operating system and install it with the guided package. On Linux you can also use your distribution's package manager (`apt`, `dnf`) or `nvm`. To check it is present: open a terminal and type `node -v` (it should respond with `v16` or higher).

No heavy infrastructure

No database (SQL/NoSQL), no cloud service, no Docker required. The app saves each project as a local file, with atomic writes and automatic backup copies: a power cut mid-save will not corrupt your work. *If you prefer containers, a ready-made Docker image is available too — see below.*

Installation in 4 steps

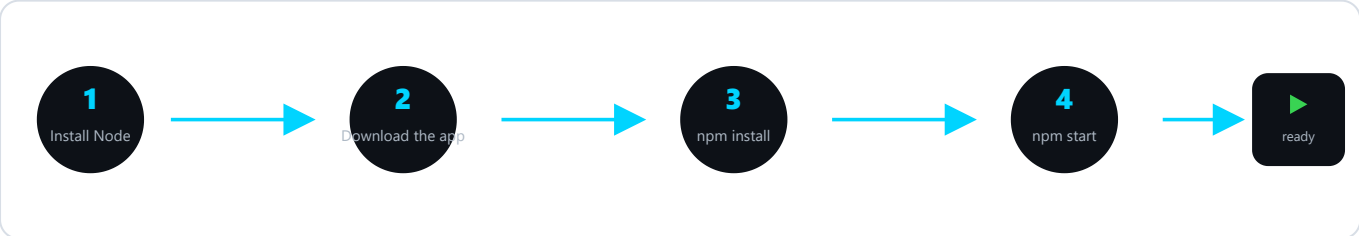


Fig. 1 — From package to first launch: four commands and the app is online.

Get the app in one of the two ways below: the first is handy if you'll update often, the second is the most immediate. Then — in **both cases** — install the dependencies and start.

Option A — with Git (recommended: updates become a single command). Requires Git installed; it downloads the project and tracks the version:

```
git clone https://github.com/muttley1973/infranetpro.git
cd infranetpro
```

Option B — download the ZIP (no Git needed). On the project page on GitHub press **Code** → **Download ZIP**, extract the folder and open it in a terminal. More immediate, but to update you'll have to download the ZIP again (see *Updating the app*).

Then, **in both cases**, install the dependencies (once only) and start. The app opens in your browser at the local address:

```
# install dependencies (once only)
npm install

# start the server
npm start

# open your browser at:
http://localhost:8421
```

> **Quick start on Windows**
>
> The folder contains `avvia.bat`: a double-click starts the server without opening a terminal. Handy for leaving the app running on a service workstation.

Alternatively: run with Docker

If you already use containers — Docker on your computer, or a NAS/server managed with **Portainer** — the project ships a ready-made `Dockerfile` and `docker-compose.yml`: the app builds and starts with a single command, and your data lives in a persistent volume that survives upgrades.

```
# set a fixed session key (once)
cp .env.example .env

# build and start in the background
docker compose up -d --build

# open your browser at:
http://<host-ip>:8421
```

The administrator password generated on first launch appears in the container log (`docker compose logs infranetpro`).

> **Network discovery & access**
> The `docker-compose.yml` uses **host networking** by default, so the container behaves like a native install — automatic **discovery sees the whole network** (MAC, vendor, SNMP) and the app is reachable at `http://<host-ip>:8421`. Since it is published on the host's interfaces (but login-protected), keep it on a trusted network; for outside access use a VPN or a reverse proxy. For an **isolated** instance (behind a reverse proxy, or on Docker Desktop where host networking is limited) use `docker-compose.bridge.yml`.

First login and configuration

- **Login.** On first launch you sign in with the default administrator account. The first thing to do is **change the password** from the user menu.
- **Configurable port.** The default port is `8421`; it can be changed via the `PORT` environment variable if already in use.
- **Access from other PCs.** Being a web server, once started it is reachable from other computers on the LAN by pointing to the server's IP (e.g. `http://192.168.1.10:8421`).
- **Where data lives.** Projects and floor plan background images are saved as files in the app's projects folder: a backup is simply a copy of that folder.

Updating the app

Your data — **projects, users and settings** — is stored in files separate from the application code: an update **does not touch it**. How you update depends on how you installed the app.

If you used Git (Option A) — just two commands in the app folder:

```
git pull
npm install
```

If you downloaded the ZIP (Option B) — download the new version from GitHub (**Code → Download ZIP**), extract it into a **new folder** and **bring your data back in**, copying it from the old folder:

Keep	What it holds
`projects/`	Your saved networks and floor-plan images
`users.json`	User accounts and passwords
`skins/`	Uploaded panel skins (if you use them)

Then run `npm install` in the new folder and restart. Do **not** copy the `node_modules` folder from the old one: it is regenerated by `npm install`.

> **Tip**
>
> Before updating, make a copy of the app folder: if anything goes wrong you can roll back in seconds. The **Git** method stays the most convenient if you update often.

02 InfraNet Pro at a glance

What it is, who it is for, and the principle that holds everything together: *manual-first*.

InfraNet Pro is designed to **draw and keep up to date** the documentation of a network: where devices physically are (the floor plan), how they are mounted in racks, how they are cabled and which VLANs they carry. It is built for people who know the network — engineers, system integrators, MSPs — and need documentation that is always accurate, not a drawing gathering dust in a drawer.

Two views of the same project

🗺 Map (physical)

Where devices are **physically** located: floor plan with rooms, positioned racks, wall sockets, access points. The reality you can touch with your hands.

🕸 Topology (logical)

How devices are **logically connected**: lines between devices derived from LLDP/CDP and switch forwarding tables, with filters for VLAN, trunk and wireless.

These are not two separate drawings: they are the **same graph** read from a physical or logical perspective. You switch between them with the `1` and `2` keys.

The principle: manual-first

> **The network suggests, the human decides**
>
> Data you enter manually is **never overwritten automatically**. SNMP discovery and consistency checking are *advisors*: they propose, flag, highlight — but the racks and cables you draw remain the authority. This is what makes the output trustworthy for a handover.

Many of the app's design choices follow from this: automatic discovery *fills in the gaps* but does not overwrite fields you have filled in; consistency checking shows you the differences but does not delete anything on its own; every alignment with reality is always your explicit decision, with a single click.

> **🔒 The lock: manual-first made visible**
>
> Next to **IP**, **hostname** and a **port's VLAN** there is a small lock. It is not a new feature: it simply makes the protection described above **visible and one-click**. **Lock open** = the field follows the network (the Sync updates it). **Lock closed** = the value is your decision: the Sync leaves it alone and *Verify documentation* warns you if reality diverges. Close it where the value *must* stay fixed (e.g. a static IP, a port's VLAN); leave it open where you want it to follow the network.

IPv6 address. Alongside IPv4, the device panel has an **IPv6** field with the **same padlock** as IPv4. On an SNMP device the **Sync auto-populates** it by reading the device's *own* address (routable IPv6, global/ULA — link-local `fe80::…` is not a management address). Close the padlock to fix it by hand: the Sync leaves it alone and **Verify warns you** if the device reports a different one. A *neighbouring* device's IPv6 can also surface from the deep scan (neighbour discovery).

03 The interface

The toolbar, the three work areas and the shortcuts that speed everything up.

The screen is divided into three areas: on the left the element **library** to drag from, in the centre the **floor plan** and **rack** views, on the right the **properties** panel for whatever you select.

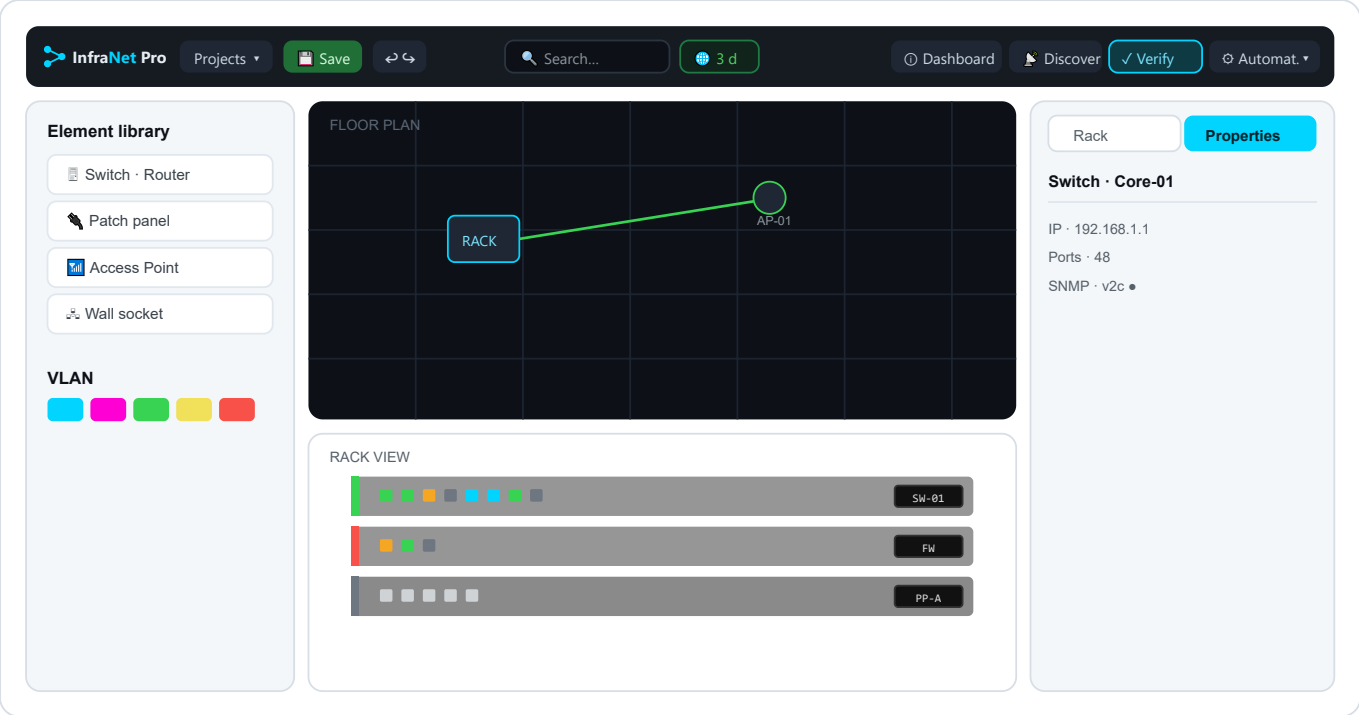


Fig. 2 — The three areas: library (left), dark floor plan + rack on white background (centre), properties panel (right). All **device data** (IP, ports, SNMP…) lives in the properties panel. In the toolbar: **Verify** is the single primary action, a **freshness chip** (🌐 3 d) sits next to the search box — it says *how* recent the last SNMP read is, colour-coded for freshness; *how many* devices answered is on the status bar and in the chip's tooltip — and ⓘ **Dashboard** opens the summary view. The old *Sync* button is gone: it lives inside **Automation** (and Verify includes it).

The status bar

Just under the toolbar runs a thin **status bar**, split into three parts that tell you at a glance where you are, what's worth doing next and how the network is doing.

- **Left: the path.** A trail like *InfraNet Pro / Project name / Floor plan*: it reminds you which project you have open.
- **Centre: the recommended step.** InfraNet reads the project's state and suggests the next useful move — *Discover now* if the network is still empty, *Verify now* if it is documented but not yet compared against reality, and so on. It's only a hint: the button opens the right feature, but you decide.
- **Right: three health numbers.** *Docs* (the share of addressable devices you have already given an IP), *Devices* (how many pieces of equipment you have documented — rooms don't count) and *SNMP* (a coloured dot: green if all respond, amber if some are missing or not polled yet, red only when something is genuinely down).

> **Numbers are computed, never estimated**

> The three stats come from the very same data you see in the Properties panel: they are a count, not a forecast. A project you have just opened and not yet synced shows the SNMP dot **amber** (not red): "not verified yet" is not "broken".

The Properties panel

It's the right-hand column: the place where **everything you know** about the element you selected lives. It doesn't always show the same things — it's **contextual**. Click an appliance and you get IP, model, ports and SNMP; click a port and you set state, speed and VLAN; click a cable and you set type, length and label; click an empty spot on the floor plan and the background's scale, opacity and grid appear. One column that changes face depending on what you're looking at.

The reason is the heart of InfraNet: objects on the map and in the rack stay **clean and readable** (icon, ports, LEDs, nameplate) while the **data** — addresses, make and model, notes, SNMP credentials — is gathered here. Separating the scene from the data is what turns a drawing into real documentation: what you type in the panel is your **documentary truth** (the *manual-first* principle).

- **Accordion layout.** Properties are split into collapsible sections (identity, **Network & Access**, the options specific to the device type…). They remember whether you leave them open or closed, so the panel comes back the way you had it.
- **It works with automation.** *Discovery* and *SNMP Sync* fill these same fields and add read-only *live cards* (toner, CPU, UPS runtime…), but they **do not overwrite** the values you entered by hand: automation enriches, it doesn't command.
- **It's what the check compares.** *Documentation check* compares exactly these fields against what's found on the network; the VLAN filter and colours also read what you set here.

> **Rack or Properties**
>
> Top-right you switch between the **Properties** panel and the **Rack** view with the `P` and `R` keys: they're two faces of the same right-hand column.

Essential shortcuts

Key	Action	Key	Action
`1` / `2`	Map / Topology view	`Ctrl+S`	Save project
`R` / `P`	Rack / Properties panel	`Ctrl+Z` / `Ctrl+Y`	Undo / Redo
`Space` +drag	Pan the map	`Del`	Delete selected element
`right-click`	Start / finish a port connection	`Esc`	Cancel connection / close panel

> **Connecting two ports — use the right mouse button**

Cables are drawn with the **right mouse button**: right-click on the source port, then right- click on the destination port → the cable is created. `Esc` (or a click on empty space) cancels. For a *cross-rack* connection, a banner advises you: navigate to the other rack and right-click the destination port.

Tip

After clicking a port, press `R` to jump to the connected rack without touching the mouse. And `Space`+drag is the quickest way to navigate large floor plans.

04 Device types

Over 30 ready-made types, split between rack appliances and floor plan endpoints.

Every element is dragged from the left-hand library. Types come pre-configured with an icon, port count and sensible attributes: a patch panel starts with 24 ports, a switch with 24, a server with a 2U height. Everything remains customisable.

Rack appliances

Type	Default	Type	Default
Switch	1U · 24 ports	Server	2U · 4 ports
Router	1U · 8 ports	Storage / NAS (SAN/RAID)	2U · 4 ports
Firewall	1U · 4 ports	UPS · PDU	power continuity / distribution
Patch panel	2U · 24 ports	KVM · Console server	out-of-band access
WLAN Controller	AP management	PBX / Phone system	telephony
Blank panel · Cable management	structural elements	Media converter	copper ↔ fibre

Floor plan endpoints

Type	Use	Type	Use
Wall socket	termination point (auto-named WA-01…)	PC / Workstation	user workstation
Access Point	Wi-Fi coverage (auto-named AP-01…)	VoIP phone	telephony, often with cascaded PC
Network printer	endpoint with IP	Smartphone / Tablet	mobile endpoint (Wi-Fi / cellular, BYOD/MDM)
Webcam / CCTV	video surveillance (auto-named CAM-01…)	Desktop NAS	Synology/QNAP, SMB/NFS/iSCSI
IoT device	sensors, embedded controllers	Smart TV / Media player	AV / signage

Projector	meeting room	Badge reader	access control
Room/Area	resizable structural element	Custom endpoint	uncovered cases

Auto-naming

Wall sockets, access points and cameras number themselves as you drag them in: no need to rename fifty sockets one by one.

05 The floor plan

Set the physical scene: import the building plan, position devices, create rooms.

The floor plan is your physical canvas. You load the building plan as a background image, scale it to the real dimensions and place devices and rooms on top of it.

Importing and scaling the background

- You load an image (**PNG, JPG, GIF, WebP, SVG**) from the toolbar. PDF is not supported as a background: convert it to an image first.
- You adjust **scale** and **opacity** of the map from the properties panel, then press **Lock scale** to avoid accidentally moving it while you work.
- The **grid** aids alignment: devices snap every 20px (snap-to-grid). You can hide it — and with it hidden snap becomes free to the pixel.

Positioning and connecting

Drag devices from the library onto the correct areas of the building plan. A **click** selects, a **double-click** opens the full properties. Double-clicking a wall socket or an AP takes you directly to the rack of the connected cable, with the path highlighted.

> **Import a VM into its host**
>
> Sometimes a virtual machine shows up on the network as its own device. Open the properties of a *hypervisor* or a *home-lab*: the whole **Virtual machines** section is the drop area (the dashed invite sits above the list). **Drag the device onto it** and the VM is absorbed into the host, inheriting its name, IP and MAC (when known — a MAC *or* an IP is enough, so it also works for devices monitored via SNMP on another subnet), and disappearing from the undocumented list. The absorb fires **only if you release inside the section**: anywhere else the device stays on the floor plan (or snaps back to where it started), and it's undoable with `Ctrl+Z`.

> **The virtual machine card**
>
> In the host panel the VMs are **a list**: one row per machine, with the state dot (green running, red stopped), the name and, in the space beside it, what tells one VM from another — role, **resources** and address. Resources show the *measured* figures when the VM has been read over SNMP (with the dish icon), otherwise the ones you declared. The row is there to *find*, not to fill in: click it and the **full VM card** opens, with the identity (name, role or service provided, guest OS — pick one from the list or choose «Custom…» and type your own), the **state at the top next to the title** (one click to start or stop it), the **allocated resources** (vCPU, RAM, disk), its **network cards** (see below) and the **handover** data: who owns it, how critical it is, how the backup protects it, plus notes. The arrow at the top takes you back to the host. All of this is information **you declare**: no network discovery can deduce it, and what you leave empty stays empty in the documents too. «Network & access» holds the name the machine answers to plus the **Management** row (protocol + address + open) exactly like a PC: leave the URL empty and it is built from the protocol and the VM's address.

vNIC ports: when a VM has more than one network card

A virtual machine can have **several cards**: a virtual firewall has the leg towards the Internet, the one towards the LAN and possibly a DMZ; a server often has a production card and a backup one. In the **vNIC ports** accordion you add as many as you need, and for each you declare a name, IP address, VLAN, MAC, IPv6 and the **port group / vSwitch** it sits on. The first row is already there: typing into it is what creates the card. The last one cannot be deleted (you empty its fields instead): a VM must always have somewhere to write an address.

Why it pays to fill them all in: **each card's VLAN** joins the trunk that the host uplink carries, **each MAC** makes that leg recognisable to the Verify pass (with a single card documented, the others would keep showing up among the undocumented devices at every check) and **each IP address** joins the duplicate audit. In the handover dossier the machine stays a single row, with all its addresses and all its VLANs listed.

One thing you will *not* find: the cable. A virtual card has none — it plugs into a vSwitch port group and rides the host's physical uplink, which is already a documented, already cabled device. For the same reason you are not asked which physical port it leaves from: if the host has two uplinks in teaming, which of the two carries that VM's traffic is decided by the balancing policy, and that is not something anyone can know — InfraNet never writes down what it cannot verify.

Reading a VM over SNMP

Many virtual machines (Windows Server, Linux, appliances) expose an **SNMP agent on their own address**: from the *Integration* section of the card you query them like any other host, with the **very same fields as a device** (driver, host override, port, timeout, community or the full SNMPv3 block with user, protocols, passphrases, security level and context) and the **Read over SNMP** button. Leave the host override empty and the VM's IP is used. What comes back — system name, the cards' MACs, how long it has been up, cores, RAM, disks — appears in a **«measured» box stamped with the date and time of the reading**, kept separate from what you declared: it does not overwrite your fields, a dedicated button copies it across (undoable with `Ctrl+Z`).

The **MAC** deserves a note: it is what makes the VM «documented» for the Verify pass, and it matters most for machines imported from another subnet, which never carry one (ARP does not cross the router). It is written in automatically **only in the case with no ambiguity**: one card measured, one card declared. With several cards the query *lists all* the MACs it measured but does not assign them, because it cannot tell which belongs to which — interfaces read over SNMP do not carry their IP address, so the pairing is yours to make by copying them onto the right card. And a MAC you already typed is never overwritten.

Two honesty rules, the same as the presence colours: if the VM **answers**, that is proof it is running and the state records it as a measurement; if it **does not answer** nothing is concluded — the outcome is noted in amber and explained, but the VM is *never* declared stopped, because it may simply have no agent, use a different community, or be filtered.

Rooms and areas

A *Room/Area* is a resizable coloured rectangle (with an anti-move lock) that you use to draw offices, server rooms and departments. It always sits beneath other elements, like a background.

Navigation	How
Pan the view	Click+drag on empty area, or `Space` +drag anywhere
Zoom	Scroll wheel (centred on cursor) or `−` / `+` buttons · range 5%–500%
Deselect / clear VLAN filter	Click on empty area

06 The rack

The realistic 19″ cabinet front: units, ports, available capacity, gateway.

Every project can have multiple racks (from 6 to 60U, default 42U). The front panel is drawn realistically — grey chassis on a white viewport, faceplates with ports and LEDs — so the rack page looks like a real photo of the cabinet.

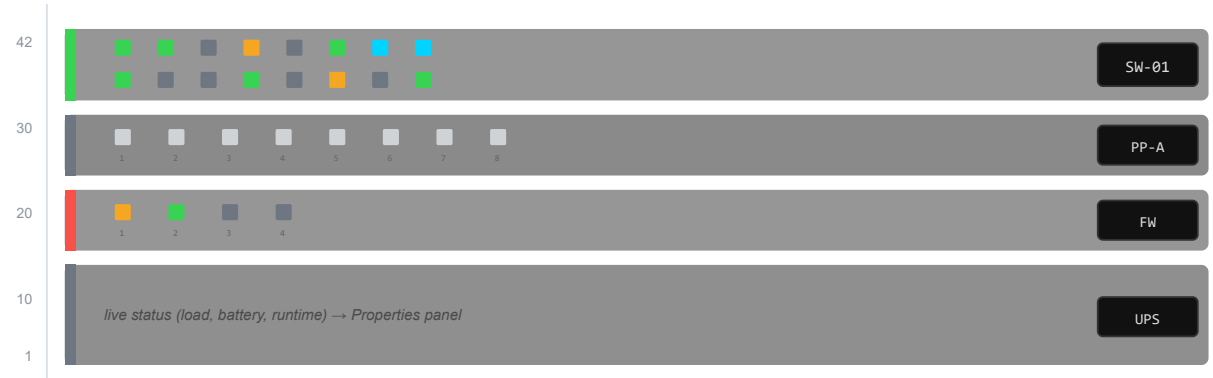


Fig. 3 — Rack front on white viewport. The **vertical stripe** on the left is the SNMP status (green ok · red error · grey not configured). Port LEDs are **small squares** (green active · amber idle · grey off · red error · blue LAG) with the **port number** below. Device data (IP, status, UPS runtime…) is read in the **Properties panel**.

Creating a rack and placing it on the floor plan

Every project can have several cabinets. Open the rack panel (the `R` key): from its ⋯ menu use **New rack** to create one — give it a name and it starts at **42U**. When you have more than one, the dropdown at the top of the panel switches between them: each rack is its own page.

- **Height in units.** The *U* field sets capacity from **6 to 60U** (default 42). If you shrink it, the app checks that the appliances already mounted still fit and warns you instead of cutting them out.
- **U numbering.** *U numbering from top / bottom* only changes how units are **labelled** (1 at the top as in telco/ETSI racks, or 1 at the bottom): handy to match the numbers on the real cabinet. It moves nothing.
- **Rename rack** and **Delete rack** are in the same menu; deleting a rack also removes the appliances it contains.

The **“Place on floor plan”** command drops the cabinet onto the map as a **marker** — a server icon with the number of appliances inside — centred on the current view and snapped to the grid. **Drag** it where it actually stands in the room; a **click** on the marker opens that rack's inside view. **“Remove from floor plan”** takes it off the map: the appliances stay in the rack, only the marker disappears.

> **Why place it on the floor**
>
> With racks on the floor plan the map tells you *where* the cabinets are in the building, and the cables running **rack to rack** become visible runs on the floor — no longer just abstract links between two pages.

Populating and configuring

- Drag appliances into the rack: they position themselves in the lowest free unit. Alternatively **import a CSV** and populate 50 appliances in a few seconds.
- The **port layout** (count, rows, separate SFP block) is adjusted from the properties panel; the front adapts from 8 to 48 ports while remaining readable.
- **Apply model.** In *Port layout*, search a real switch/router model and apply it in one click to set **ports, SFP/QSFP and MGMT exactly**: the front is redrawn by the same renderer as native devices, so it's identical. The catalog is generated from public (CC0) device-model data — only the data; the artwork stays yours and live. Up to **24 SFP per block**.
- **Click** on a port = configure it; **right-click** = start a cable to another port (even on a different rack).

Free ports and L3 gateway

🟢 Free ports

Answers the everyday question: "*where do I plug in the new device?*". Highlights free ports **directly in the rack** (green = free, yellow = free on paper but seen active via SNMP) and produces a report with totals, exportable as CSV.

🌐 L3 map / Gateway

Promotes the gateway from a mere IP to a **"network → routing device" relationship**. One row per **declared network** — IPv4 and IPv6, including those that carry no VLAN — with the device answering that address and the suspicious cases: gateway with no device, outside its own network, of the wrong family. An **L3** badge marks devices acting as gateways.

07 Cables, connections and wireless

Clear connections at a glance, the cable editor, the complete physical path and Wi-Fi "wave" associations.

A cable is created with the **right mouse button**: right-click on the source port, right-click on the destination port. The colour follows the VLAN; the **style** indicates how reliable that connection is.

The visual language of cables

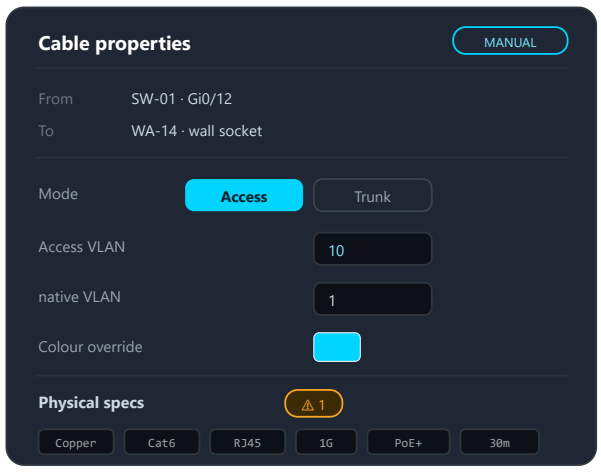
Appearance	Meaning
Solid coloured line	**Manual** or **discovered** via LLDP/CDP — reliable
Animated dashed line	**Inferred** from MAC/ARP/FDB tables — to be confirmed (may be a transit link)
Sine wave	**Wireless connection** (radio association)

Deduced uplink to a "blind" switch

When Sync finds a *documented* switch's MAC learned on another switch's port, it attaches the uplink cable even if the downstream switch does not answer SNMP (e.g. a Zyxel GS1200): we know *that* it sits behind that port, not *which* of its ports the cable enters. The cable is therefore born **"Inferred · to verify"** — even when the local port is a **LAG member** — and only **one** cable is attached: confirm it and fix the real downstream port by hand (plus any other LAG members). It is never shown as a confirmed "LAG", because the downstream aggregation was never observed.

The cable panel: what you can set

Selecting a cable turns the right-hand panel into its **editor**. Here you declare what it is made of and what it carries — always following the *manual-first* principle: what you write by hand is authoritative.



- Access or Trunk with one click — the right field appears
- The VLAN you enter here is authoritative (manual-first)
- Override: force a colour on the cable
- Physical specs with validation that explains why
- Double-click the cable → physical path (Fig. 5)

Fig. 4 — The cable editor in the Properties panel: endpoints (read-only), **Access/Trunk** mode, access VLAN and native VLAN, colour override, physical specs with the ⚠ warning badge, and (for radio links) the wireless toggle.

- **From / To** — the two endpoints (node and port), read-only.
- **Access or Trunk mode** — *Access* = a single VLAN (untagged); *Trunk* = multiple tagged VLANs (e.g. `10,20,100-200`). The right field appears depending on the choice.
- **VLAN** — the access VLAN (or the list of VLANs carried on the trunk) plus the editable **native VLAN** (PVID).
- **Colour override** — forces a colour for that cable, overriding the automatic VLAN colour.
- **Physical specs** — Medium, Category, Connector, Speed, PoE, Length, with the consistency validation described below.
- **Wireless connection** — the 📶 toggle turns the cable into a radio association (wave); with wireless enabled, the physical specs disappear.

The physical cable path

A real connection is often a **chain**: the switch reaches the patch panel, the patch panel reaches the wall socket, the socket reaches the endpoint. With a **double-click on a cable in topology** the app highlights *the entire physical path*, not just the clicked segment, and shows every segment in a single panel where you can select and edit each one individually.

Fig. 5 — **Physical path** highlighting (routing mode): endpoints in yellow, trace in cyan, unrelated branches dimmed. Press **Esc** to exit.

Smart validation (warnings, not blocks)

In the cable's physical specs the app checks that medium, category, connector, speed, PoE and length are consistent — and **explains why** when something does not add up:

- 10G on Cat5e → Cat6A required. On **Cat6**, 802.3an/TSB-155 allows 10G **up to 55 m**: if you declared a length within that, there is no warning — the run is compliant, and flagging it would only teach you to ignore warnings;
- copper run over 100 m → outside TIA-568 standard, use fibre;
- **fibre run beyond the reach of its optical class**, at the declared speed. Note the point that matters: on multimode the distance *drops* as the bitrate rises — OM3 carries 300 m at 10G but only 100 m at 40G, OM4 reaches 400 m at 10G. Class, speed and length all have to be declared: with any of them missing, the app says nothing;

- PoE on fibre → fibre does not carry power;
- medium↔connector inconsistencies (RJ45 on fibre, LC on copper).

These remain **warnings**: documentation stays free (manual-first), but you know immediately what and why.

Wireless: the radio port

> **"Wave" associations**
>
> Wi-Fi devices (APs, routers, firewalls) expose a 🔁 **antenna badge** 📶: it is the *radio port*, a virtual connector that hosts many clients without occupying physical ports. Drag a connection from the client to the radio badge and it is created as wireless, drawn as a wave. An AP can broadcast multiple SSIDs, each on a different VLAN: clients land automatically on the VLAN of their SSID, and the AP's wired uplink becomes a trunk carrying all those VLANs.

> **AP mode — capability, not role**
>
> SSID fields appear only on devices that *are* access points by type (AP, router, firewall). A Wi-Fi-capable device that isn't one — a PC or server used as a **hotspot** — can turn on **"AP mode"** in *Network & Access* (next to "Wi-Fi capable", on the same row) and broadcast an SSID **without changing its type**: it stays a PC, but with the SSID editor unlocked. It's opt-in (off by default) and reversible — turning it off removes any orphaned BSS.

The cable's proof-state

After a **Documentation check** every cable inherits the *reachability* of its two endpoints and shows it with a **badge**, next to the provenance ones (LLDP/CDP/MAC), of the same size and shape:

- `Fresh` / `Weak` — an **inferred** cable with recently confirmed endpoints: a strong adjacency (LLDP/CDP) or a weak signal (FDB/MAC/ARP);
- `Ghost` — an inferred cable whose evidence has **vanished**: the endpoint no longer answers, or the confidence has decayed over time. It stays drawn, never deleted;
- `Declared` — a **manual** cable: it stays declared even when the device goes quiet. Physical cabling is not reachability — if the switch is off the cable is still in place, and the "it's down" signal belongs to the *node*, not the cable;
- `Review` — reality **contradicts** that cable: the port announces a different LLDP/CDP neighbour than the one cabled, or the endpoint has changed its IP or hardware identity.

> **The ghost is visible, not hidden**
>
> On the map an inferred cable is **dashed** and a ghost is **faded**; a declared cable stays solid. In the *Dashboard* the "Cables" tile sums it up — "N ghost · N inferred · N declared" — the cabling's **identity-in-numbers**. Badges appear only *after* a check: on a network never verified, no ghosts are invented.

08 Networks, VLANs and colours

One colour per VLAN: cables colour themselves automatically and the network is readable at a glance.

You assign each VLAN a number, a name and a colour. Cables colour themselves automatically based on the VLAN they carry: a well-structured network becomes readable at a glance.

Default palette

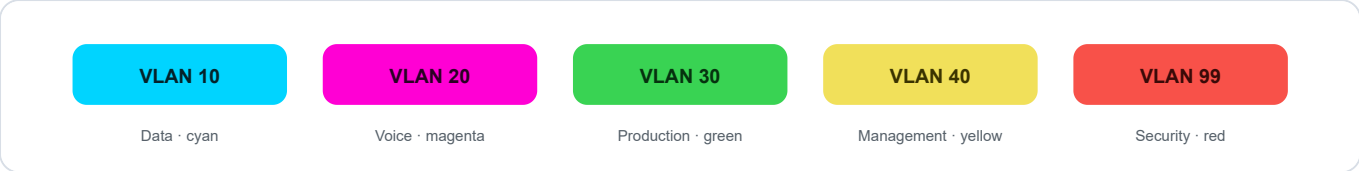


Fig. 6 — The five default VLAN colours. New VLANs receive a distinct colour automatically; every colour can be changed.

Access and Trunk

Mode	Meaning
Access	The port belongs to a single VLAN (untagged traffic). Typical for a PC, printer or camera.
Trunk	The port carries multiple VLANs (802.1Q-tagged): e.g. `10,20,100-200`. Typical for inter-switch uplinks and multi-SSID APs.

The app **propagates** VLANs automatically along cables and through pass-through elements (patch panels, wall sockets, media converters), so a run *switch → patch → socket → device* reflects the same trunk. The mode is a property of the active interface (the switchport), as in reality.

The VLAN filter

In topology, clicking a VLAN badge in the legend **isolates** that VLAN: everything else dims or disappears, and you see only where it goes. A double-click opens the member list. This is the quick way to verify that every VLAN is cabled where it should be.

Declared networks

A VLAN and a network are two different things, and the app keeps them apart because in reality they are: a **VLAN** is a broadcast domain — a number, a name, a colour; a **network** is an address space — a prefix (e.g. `10.0.10.0/24` or `2001:db8:0:14::/64`), a gateway, a DNS. A VLAN can carry more than one network, and a network need not have a VLAN at all.

- **Dual-stack.** The same VLAN can hold its IPv4 *and* its IPv6, each with its own gateway — because in reality those are two different gateways.
- **Secondary address.** The network someone added a second range to over the years because the `/24` ran out: two prefixes on the same interface, which is legitimate as long as they do not overlap.

- **Networks with no VLAN.** A point-to-point link between two routers, a transit network, out-of-band management: real addresses that have no VLAN.

Where you declare them: one place

Networks are written **from the "Networks" section** and nowhere else. The reason is in the model: a network has **at most one VLAN**, so that field exists on one side only — on the network. Being able to write it from the VLAN card too meant giving an "add/remove" to a single-valued field, and that is where the confusion came from between taking a network off a VLAN and deleting it from the document.

- **In "Networks" you add, change and delete.** The field at the foot of the list declares new networks (with no VLAN); click a row and in the detail you pick its VLAN from the dropdown, or "none". The **×** deletes the network from the document.
- **The VLAN card shows and takes you there.** It lists the networks that reference it — at a glance, while you are looking at the VLAN — and clicking one takes you to its row under "Networks", already open. It changes nothing. (The one thing it does write is the *gateway device*, because that really is an attribute of the VLAN: it is the interface that routes it.)

You can type **several networks separated by commas** and they all land in one step (one Undo takes them all back). Anything not recognised **stays in the field, in red**: it does not vanish and it is not guessed at.

The "Networks" section: the addressing plan

"Networks" is **the first section** of the project context — before "VLAN", because the addressing plan is the document and a VLAN is a label a network *may* carry — and it is **a single list**: every declared network, with a VLAN or without, one per row, **sorted by address**. The columns are named: `NETWORK`, `VLAN`, `USAGE`. The VLAN is a pill with a dot in its colour, not a container. Overlapping pairs are marked in red with the conflict spelled out underneath. Each row opens its own detail; the field for adding one sits at the foot, after the list.

> **The bar next to the fraction**
>
> Every row carries a **miniature occupancy bar**: the same measurement as the "Occupancy" block in the detail, same colours and same amber threshold. It is there because six fractions with different denominators — `2/254`, `21/254`, `0/126` — do not compare at a glance, while six bars do: the question the list answers is "where am I running out of room". On IPv6 there is **no bar**: 2^64 addresses is not a percentage, and drawing a track under a denominator that does not exist would be an invented figure.

> **Why by address and not by VLAN**
>
> Two networks that tread on each other are neighbours in the *address space*, not in the VLAN list: sorting by VLAN hides exactly what you need to see. An IPv4 and an IPv6 on the same VLAN are not a conflict — they are dual-stack, and different families never intersect.

The **gateway address** finds its network by **containment**: if it does not fall inside the prefix the app does not guess — it flags it in red and says which network it actually falls in. The **DNS** stays an ordinary field, because `1.1.1.1` does not sit inside the network it serves and there is nothing to infer.

Two gateways that are not the same thing

Two related fields appear in the panel, and it helps to know which question each answers.

- **Gateway address** (in the network detail, under "Networks") is `192.168.20.1`: what clients set as their default gateway. It is *per network* — a dual-stack VLAN has two, one IPv4 and one IPv6.
- **Routed by** (on the VLAN card) is *which device* in the project routes it: the box you log into when that VLAN stops passing. It is *per VLAN* — there is one SVI, even when it carries two addresses.

The app tries to connect them by itself: if a documented device holds that address, "Routed by" says so — *"RT-Edge, inferred from address 192.168.20.1"* — and asks you to confirm. Often it cannot, **for a real reason**: in InfraNet a device carries its *management* address (`10.0.0.5`) while the VLAN's SVI answers on `192.168.20.1`. Same box, different address, no match possible — so you pick it by hand, and that choice wins and is never overwritten.

> **Where the warning appears**
>
> "No documented device holds this address" reads **next to the address**, in the network detail: that is where you fix it. Not on the VLAN card, which does not even show that address.

> **If you import from NetBox**
>
> In NetBox a prefix's VLAN is **optional**, and on a real installation most prefixes have none. They all come in now, and the import preview says how many they are and which VLANs carry more than one network.

Address occupancy (IPAM)

If you've loaded the **DHCP leases** (see *Documentation check*), the card shows the **real address occupancy**: how many addresses are used out of the total capacity, with a bar that turns **amber** when the subnet is nearly full, and the split between **documented**, **DHCP-only** and **free** addresses.

> **On IPv6 there is no percentage**
>
> A `/64` holds 2^{64} addresses: "3% full" would mean nothing, and a bar under a denominator that does not exist would be a lie. For an IPv6 network the app counts the addresses it has **seen**, and stops there. Two spellings of the same address (`2001:DB8:0:20:0:0:0:1` and `2001:db8:0:20::1`) are *one* address, not two: the comparison is on the canonical form, not on the text.

> **From leases to the map in one click**
>
> When there are addresses *seen by DHCP but not yet documented*, the card shows **"N undocumented → Adopt"**: one click takes those devices (with MAC, IP and hostname) straight into the Adopt window, so they enter the map already documented.

A network is a thing of its own, not a field of the VLAN

Up to 2.8 the subnet was an attribute of the VLAN: one VLAN, one address. That is how one starts thinking about it, but it is not how real networks are built. A VLAN can carry two networks (dual-stack IPv4 + IPv6, or a second range on the same SVI), and a great many networks have no VLAN at all — on a real NetBox they are the majority.

From 2.9 the **network is first-class** and the VLAN is a label it may carry. The **Networks** section lists them all, sorted by address, with overlaps spelled out. You can add several at once, fix an address in place, delete a row or clear the plan — both destructive actions ask first.

> **Project format.** The file moves to *schema 2* and is migrated in place on open: there is nothing for you to do, and older projects still open.

What changes in practice: the VLAN card *shows* its networks and takes you to them, but no longer writes them — the record lives in Networks. The two gateways finally have distinct names: **"Gateway (IP)"** is the address and sits on the network, **"Routed by"** is the device and sits on the VLAN. The L3 map is one row per network, not per VLAN.

IPv6 can be declared, not only measured: a bare address normalises to its /64, and on a /64 InfraNet shows the addresses actually seen instead of a percentage that would mean nothing. In the REST inventory networks come out as `prefixes`, with VLAN (`null` when there is none), name, gateway, DNS and provenance.

> **A declared network is recognised by its real shape.** Verify and the Dashboard each read a CIDR their own way, and both IPv4-only: a declared IPv6 network never earned its badge, and "declared if it sits in a subnet of /24 or wider" was a threshold left over from when every row was a /24 — a /22 or a /25 was judged against 24 instead of its own size. One reader answers now, and a row is declared when a declared network contains it and is no narrower.

09 Port states and presence

LEDs, the SNMP stripe and the dimming of devices that have disappeared from the network.

Status is read from colours, everywhere: on port LEDs in the rack, on device borders in the floor plan and in exports. They are the same four colours as the app.

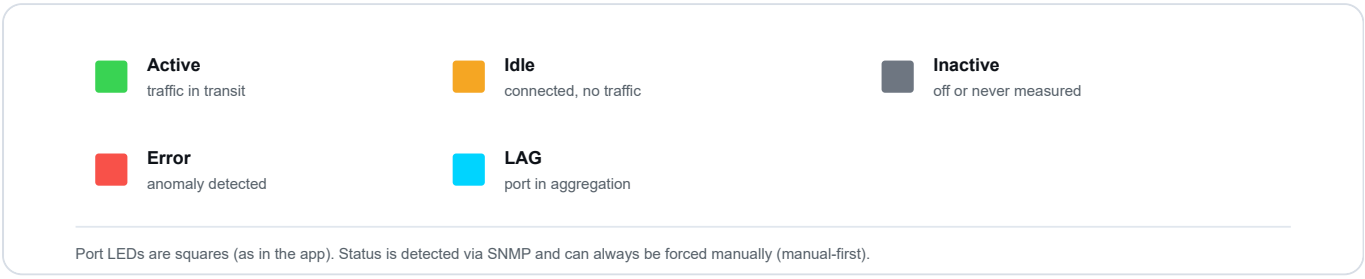


Fig. 7 — Port colour legend, identical in rack, floor plan and export.

> **On a passive hop, "active" means occupied**
>
> A patch panel, a wall outlet or a media converter never sees traffic go by: they are copper. There, green says **there is a patch cord in it**, not “traffic passing”. The state sets itself when you connect the cable and clears when you remove it; the *Status* field in the panel stays editable so you can state what the app cannot see — a cord already run but not yet documented, or a faulty port. Speed and VLAN are not offered, because those really are decided by the device upstream.

The address belongs to the port, not to the device

A port's card carries an **Interface address**. It is for the devices that hold **one address per port** — routers, L3 switches, firewalls: the address facing the LAN is one thing, the one facing the WAN link is another. The address you *administer* the device on stays where it was, on the device card.

It also stops you seeing double: a router that announces itself over LLDP with its other interface's address looks, at first glance, like a second device. Declare that address on the right port and the document tells the truth — one device, several networks.

> **Not offered on passive ports**
>
> A patch panel or a wall outlet has no address: it is copper passing through, not an interface terminating traffic. The field does not appear there, so nothing untrue can be written into it. ⚠ What you type is stored **as typed**: if it does not look like an IPv4 address the panel says so, but does not refuse it — this is your statement, not a form to fill in.

The SNMP stripe ≠ presence

> **Two different signals**
>
> The **vertical stripe** on the left of the device indicates whether the last SNMP poll succeeded (green) or not (red/grey). This is different from *network presence*: an endpoint that does not speak SNMP (a PC, a camera) is

always "grey" on the SNMP side, even if it is powered on and working.

> **On the floor plan: orange ring ≠ red halo**
>
> On the map a device that **will not answer the SNMP query** wears an **orange ring**; one the Verify has **proven absent** wears a **red halo**. Same shape, different colour, because they are different pieces of news: "it is there but will not answer me" is not "it is gone". The orange has three usual causes — a wrong community or v3 credentials, an ACL that excludes the server, a stopped SNMP agent — and hovering the device reminds you of them. ⚠ The orange ring does **not** appear on a device the Verify could not reach: there SNMP fails for that very reason, and calling it a fault would be an alarm with no evidence behind it.

Absent devices, dimmed on the map

When **Verify documentation** finds **no signal** from a device (no SNMP, no ping, no ARP, no trace in switch tables), that device is **greyed out** on the floor plan and in the rack: it remains clickable, but at a glance you understand that "it is on paper, but not on the network". When it comes back online, it lights up again automatically. Devices that are **Not verifiable** (on a subnet the check could not reach) are **not** greyed out: since their absence was never confirmed, they stay normal.

> **This is a Verify documentation feature**
>
> The dimming is based on the **full multi-signal sweep** (ping/ARP/TCP in addition to SNMP and forwarding tables) performed by **Verify documentation**: it is that process which determines absence and applies the greying.

Shut by hand, or merely without link?

These are two different facts, and for a long time InfraNet read only one of them. A switch exposes *two* states per port: whether there is link (`ifOperStatus`) and whether someone turned it off (`ifAdminStatus`). The first is a symptom — device off, dead NIC, SFP pulled, cable gone: we do not know which. The second is a **decision**, written on the device by a person.

On a real switch the difference shows at once: eight ports "down", and no way to tell the two someone had shut from the six that were simply empty. From 2.9.2 the two are separated, on a **greyscale** rather than with new colours — green, red and amber already mean something else:

- **Near-black** — the port is in `admin shutdown`.
- **Dark grey** — no link across three consecutive Verifies. Measured, but ambiguous: the threshold is there so a reboot does not raise an alarm.
- **Light grey** — nothing is known. Which does not mean "off".

The same colours reach the **PDF dossier** and the **draw.io** export: paper has no tooltip to ask, and that is where a wrong colour lasts longest. In the port's **Properties** the "SNMP" line says it in words, next to what the device reports.

> **Your word stays yours.** If you declared the port's status yourself, the LED shows what you chose: the measurement does not overwrite it. Where a declaration and a measurement disagree is the Verify, not a 7-pixel square. And the reading **expires**: if the switch stops answering, the port goes back to grey — an assertion this strong must not outlive the evidence that carried it.

On cabling: a **cable you declared yourself** running to a shut port gets a **"Port shut"** badge and a row of its own in the Verify — and their count appears in the Dashboard's «Cables» tile, next to ghost, inferred and declared — because your word and the word written on the device contradict each other — usually it means the kit was decommissioned and the document has not caught up. An *inferred* cable on that port, on the other hand, does **not** become a ghost: through a shut port the link never forms, but that "no link" is the consequence of a decision, not an observation about the cabling.

What counts as a measurement

The dark grey "no link" is earned over three Verifies, and none of the three may be a pretend one. These do not count:

- **Verifies run while the port was shut.** While a port sits in `shutdown` we are not looking at the cable, we are looking at a choice. When you reopen it the port goes back to **light grey** and the dark grey has to be earned again from scratch — it used to drop straight back, on the strength of readings that had observed nothing.
- **Readings that never arrived.** If the device answers "I cannot tell" (`ifOperStatus` has a value for exactly that, and the port used to light up *amber* as though it were under test), or if the port does not appear in the answer at all — which happens with long tables that get cut short, or when a module is pulled — the previous measurement is **forgotten** instead of standing in for it. A port we know nothing about does not age towards "no link".

> **If you open a project saved earlier**
>
> The link measurement is a new field: a document saved by an earlier version does not carry it, and until you run a **Sync** no port accumulates. That is deliberate — about those ports we do not know anything yet.

10 SNMP: discovery and synchronisation

Letting the network speak for itself — with SNMP v1, v2c and v3, even without credentials on the first pass.

InfraNet Pro speaks **SNMP v1, v2c and v3** natively. Configure a device with a driver and address, and the app reads interfaces, VLANs, port status, MACs, and — on devices that expose them — printer toner levels, CPU/RAM, UPS and power continuity status.

Three actions that look similar but answer different questions

	🔍 Discover	⟳ Sync	✓ Verify
Question	"What is on the network that I have not yet drawn?"	"How are the devices I have drawn doing right now?"	"Does my drawing match reality?"
Starts from	a range/subnet	already configured IPs	already configured IPs
Finds new devices?	**Yes** (that is its purpose)	No (at most adds cables)	No (flags them, does not create them)
When	first mapping / new devices	quick refresh of status and topology	audit "is the doc still accurate?"

In the toolbar they are not three equal buttons

The three *questions* stay distinct, but on the toolbar there is only one primary action: **Verify**. One click re-reads the SNMP data (**it includes Sync**), rebuilds the topology and *then* compares against reality. The **bare Sync** — just a status and cable refresh, without the comparison — lives in the `Automation` menu (handy for a light refresh); **Discover** keeps its own button. A **freshness chip** next to the search box always reminds you how recent the last read is.

The most common confusion: Discover ≠ Sync

Discover starts from a *range* and looks for new hosts (active scan). **Sync** starts from the *IPs you already have* and refreshes them, also deriving cables from forwarding tables (FDB) and LLDP/CDP: you do not need "Discover" to get connections, **Sync already collects the FDB** and builds the topology on the fly, in one shot.

Discovery: scanning a subnet

1. You specify the subnet (e.g. `192.168.1.0/24`) and the SNMP credentials.
2. The app scans IP by IP, **classifies vendor and type** from the device profile (firewall, UPS, NAS, server, router, switch) and presents the candidates.
3. You select which ones to import: they enter the rack already with basic data filled in.

Scan options. Under the fields you set the **Speed** and a **"Search deeper"** box of four options (all *opt-in*): they find more at the cost of time, and stay separate because some add "noise" on the network.

- **Speed** — the probe pace toward *unknown* hosts: **Fast** (parallel), **Balanced** (gentle pace, the default) or **Careful (monitored networks)** (serialised sweep with **random order** and random pauses/*jitter* — nmap's *polite/T2* profile: documenting the network **does not trip its IDS/IPS**, but is slower). Polling of already-known devices stays parallel (it is not a scan signature). **Fast and Balanced** also resolve the **NetBIOS names** of Windows PCs (one `nbtstat` query per alive, nameless host); **Careful** does not, to add no NetBIOS footprint.
- **Recognise devices better** — probes the most common TCP ports on candidates (HTTP banner, Google Cast, RTSP) to refine type and vendor.
- **Also find quiet devices** — listens to the local segment's mDNS/Bonjour and SSDP/UPnP announcements: it recognises phones, TVs, printers, smart appliances and IP cameras (via ONVIF) that **ignore ping/SNMP** — a *measured*, vendor-neutral type and model. Same subnet only (link-local multicast).
- **Include hosts that ignore ping** — SNMP-probes *every* address in the range, even those silent to ICMP: useful behind firewalls or CoPP that block ping but let SNMP through (an SNMP responder is marked alive — measured proof, not invented). Slower.
- **Follow the links between switches** — from the reached devices it follows the announced *neighbours* (LLDP/CDP), widening discovery beyond the given range.

When the scan finishes the panel switches to the **results phase**: the form disappears and only the **table** of found devices remains; the **"‹ New search"** button takes you back to the form (range already filled in) to refine and re-scan.

SNMP v3 detection works even **without credentials** on the first pass (it recognises the device and indicates that authentication is needed), and handles mixed-version environments in the same scan.

Vendor recognition. Each device's manufacturer is resolved from two local, refreshable datasets: **IEEE OUI** (from the MAC) and **IANA PEN** (from the SNMP `sysObjectID`). Together they cover tens of thousands of vendors, so a new model is recognised *without* updating the app. To refresh them from the latest official registries: `npm run update-oui` and `npm run update-pen`.

Automatic monitoring (two depths)

From the **Network automations** popover, a single switch turns on background monitoring, with a chosen **depth** and one interval (adaptive to the depth). **Light** refreshes *only* the SNMP data (ports, VLANs, status) at short intervals (5–30 min; 5 min is the SNMP polling standard — below that it can't keep up on large networks), **without** touching the topology — the cheap "is it up right now?" path. **Full** runs a silent **Verify** at hourly-to-daily intervals (1h / 6h / 12h / 24h): it already includes the SNMP refresh and adds the comparison against reality plus history. The two depths do *not* run together — a Verify subsumes the poll, so you pick one — and an in-flight guard prevents two overlapping runs (and two SNMP sweeps). The manual *Sync* (same `Automation` menu) remains the way to realign connections too in one shot. Honest limit: it runs while the browser tab stays open.

> **UPS and power continuity units**

UPS and ATS units have no interfaces to map: Sync queries them for dedicated parameters and populates a "Live status" block — mains/battery power feed, percentage and remaining runtime, load, voltages. UPS OIDs are standard (RFC 1628, vendor-neutral).
For transfer switches no equivalent standard exists: we use the *APC PowerNet* profile, the only publicly documented one. An ATS from another make answers the SNMP session but not those OIDs, and in that case the card **says so** — "Not readable with this profile" — instead of staying blank as though you had never queried it. No values are invented: those fields are filled in by hand.

Sync finds wireless clients too

Beyond cables, Sync recognises **wireless associations** and draws them as **waves** (radio↔radio), not cables. It derives them from two *vendor-neutral* SNMP signals: the **FDB** (a MAC learned on a device's radio interface) and the **L3 neighbour table** (ARP/ND) — the latter universal, covering devices that don't expose the FDB over SNMP (PC/hotspots, L3 APs). It works on the **all-in-one** box (router+AP+switch): its FDB already holds the wireless clients. The discovered client gets a station radio and attaches to the AP's radio; the **BSS/SSID** is chosen by *VLAN match* (ambiguous → you pick it, no guessing). The L3 signal fires only for devices that **broadcast an SSID**, so a station is never mistaken for an AP. As always *manual-first*: a hand-drawn wave is never touched. The client must already be a node (import it from "Discover"): Sync *links*, it doesn't invent the device.

Honest note. A device that exposes SNMP on the LAN is required: the typical **ISP all-in-one** boxes (SNMP closed, Wi-Fi and LAN in a single bridge) are out of reach. Works on enterprise AP/WLC, MikroTik, UniFi, OpenWrt and PC hotspots (Windows/Linux).

Autosave

Still from the **Automations** popover, **Autosave** (opt-in, off by default) makes the work continuous: it persists fresh data on its own after any real change (an edit, a Sync, a Verify), a few seconds after activity settles — just browsing never saves. Periodic verification is not a separate switch: it lives in **Automatic monitoring** above, at depth *Full*. Each Verify appends a history row and can keep restorable "snapshots" — see ch. 17.

11 Automatic topology

The L1/L2 cross-check: the app draws discovered neighbours and proposes cables to create.

The Topology view reads the neighbours declared by devices (LLDP/CDP) and forwarding tables, and draws them as lines on the map. It is the **mirror** between the cabling you have documented and what the network reports about itself.

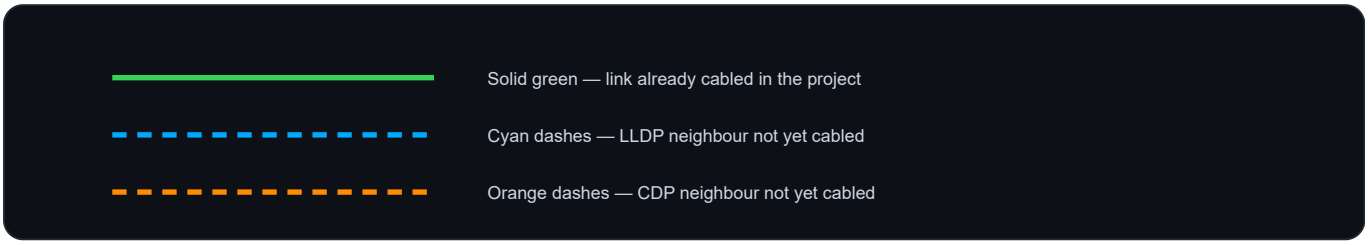


Fig. 8 — Topology lines. Clicking a dashed line opens the detail and the **"Create cable"** command, which matches ports by interface name.

Filters active only here

- **TRUNK + ACCESS** — cycles through three states on each click: *together*, *access only*, *trunk only*. Excluded lines **disappear** rather than fading: a nearly transparent line is still there to read and to reach across with the mouse. As with the medium, the pill carries the number it is hiding. A run carrying both trunk and access stays visible under either filter, because it is true under both; one discovered over LLDP and never documented has no declared mode, so it drops out of both and is counted separately — calling it “access” would be an invention.
- **ENDPOINT** — hides the last segment towards terminals: the view stops at wall sockets and the backbone (useful for reading the structure without the noise of PCs). Terminal means *it has an IP address*, so a VoIP phone counts too, even though it carries the PC downstream. Sockets, patch panels and media converters do not — they have no IP, they are copper, and they stay to mark the edge of the filtered view.
- **WIRED + WLAN** — cycles through three states on each click: *cable and waves together*, *wired only*, *WLAN only*. It is there so you can look at one medium at a time: an access point with twelve wireless clients is unreadable among the forty cables running beneath it. While a filter is on, the pill carries the number of links it is hiding.

> **Create cable from suggestion**
>
> Let LLDP find the neighbours, then click "Create cable" on the dashed line: cable the network on the map without drawing every connection by hand.

12 The workflow

Not a straight line, but a *maintenance loop*: discover, document, verify.

You import the floor plan once. Then the *discover → document → verify* cycle runs over time: that is what keeps the documentation alive instead of letting it age.

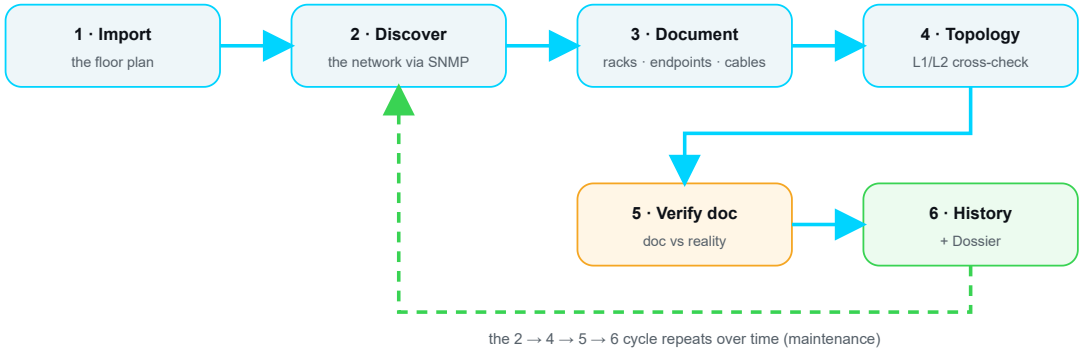


Fig. 9 — The maintenance loop. Phases 1–3 are the initial setup; **2 → 4 → 5 → 6** is the recurring cycle: after History/Dossier you return to **Discover** for the next round.

Two ways to start — and a principle that ties them together

The cycle above is the **maintenance** one. At the very start, though, there are **two paths**, both valid, depending on how the project is born:

- **Planned network (*declare-first*)** — you know your addressing plan. Then: *new project → import the floor plan →* ***declare all the networks*** *(VLANs and subnets in the panel) → scan the assigned subnets.* The plan first, then the check against reality. This is the clean path.
- **Inherited network (*discover-first*)** — you don't know the subnets up front: discover what's on the network first, and **declare afterwards** based on what you find.

> **The principle: what's declared is law**
>
> What you write in the **VLAN/subnet** panel is the **project truth**. If you declare a `/16` or a `/28`, the app measures capacity and compares reality **against that prefix** — the scan *confirms* it, it does not redefine it (it's *manual-first* applied to the addressing plan). And whatever it finds **outside** any declared subnet it doesn't hide: it flags it as **"undeclared"**, so you know what's missing from the plan. Declare-first is **recommended, not mandatory**: with no declarations the app still works (it assumes a `/24`), but tells you clearly. See the Dashboard (chapter 15).

Where manual-first lives

- **In discovery** — fields you fill in manually stay yours: SNMP fills the gaps, it does not overwrite them.
- **In topology** — it is a mirror: it shows where cabling and reality agree or diverge, but it does not touch your cables.
- **In verification** — "Update doc" is an explicit action you take; the tool flags, it does not destroy.

Where Verify lands

The outcome of a **Verify** is no longer a flash that vanishes: it **lands in the Dashboard**, in the **"Conformance"** column, and stays there as state — with the differences ready to decide (see chapter 14 and chapter 15). The *loop* never really "ends": the Dashboard tells you when it is worth starting again.

13 AI assistant (advisory)

Query your documented network in plain language — your data stays yours.

The **AI assistant** is an *advisory* chat (a copilot, not an autopilot): it answers questions about **your documented network** in plain language — presence, VLANs, free IPs, **ports** (status/VLAN/what's connected), **SNMP health** (CPU/RAM/toner/UPS) and **topology**. It lives as a **third «Assistant» tab** in the right panel (next to `Rack` and `Properties`; shortcut `A`) and a toolbar button.

> **The principle**
>
> **"InfraNet computes, the AI narrates."** The numbers (drift, free IPs, gaps) are computed by InfraNet and passed ready-made; the AI puts them into words and **cites the data**. We never ask the LLM to do network arithmetic — it would get it wrong.

Bring your own key, your choice of provider

A single hook to an **OpenAI-compatible** endpoint: **local by default** (Ollama, no key, data never leaves the machine) *or* any cloud model you choose (OpenAI, Anthropic's OpenAI-compatible endpoint, …). Configure it under `Users and access` → `AI assistant`: enable, endpoint, model, key (write-only). InfraNet **neither hosts nor pays for** inference and **bundles no model** — it stays lightweight.

How to configure — local or cloud

Go to `Users and access` → `AI assistant`, turn on **Enable**, paste the **endpoint** and **model**, paste the **key** (write-only: after saving you only ever see "configured", never the value) and press **Save**. The header chip shows where inference runs: 🔒 **Local** (green) or ☁ **Cloud** (amber).

Provider	Endpoint	Model (example)	Key
Local — Ollama (default, privacy)	`http://localhost:11434/v1`	`llama3.2:3b`	*empty*
Cloud — OpenAI	`https://api.openai.com/v1`	`gpt-4o-mini`	`sk-…`
Cloud — Anthropic (OpenAI-compatible endpoint)	`https://api.anthropic.com/v1`	`claude-haiku-4-5`	`sk-ant-…`

Local: install **Ollama** and pull a model (`ollama pull llama3.2:3b`) — no key, data never leaves the machine. **Cloud:** create the key in your provider's console and paste it here; quality goes up but **the sanitized data leaves toward the provider** → before enabling, use `Show what leaves` to see exactly what. Any **OpenAI-compatible** endpoint works (Azure OpenAI, OpenRouter, vLLM, LM Studio, …): only the URL, model and key change.

> 🔒 **Data security — by construction**

> Two guarantees, not promises: **(1) the key lives only on the server** (file `data/ai-config.json`, outside the repo, or the env var `INFRANET_AI_KEY`) and **never returns to the browser**; **(2) only the "allow list" leaves**: the context is built from the same *allowlist* as the REST API — the **SNMP community and credentials are not in the list**, so they *physically* cannot leave (plus a secret-name *denylist* on the health data). The `Show what leaves` button shows the **exact sanitized JSON** *before* you enable anything, and a guard test **fails the build** if a secret ever reached the context. Local = nothing leaves; cloud = only the sanitized data.

> **No hallucination**
>
> A hard rule in the prompt: **never invent** names, IPs, MACs, VLANs or ports. If a fact isn't there, the assistant answers "*not in the documentation*" and suggests how to find it (Discover / Verify). It is **advisory**: it proposes, **you confirm** (manual-first); any Ansible playbook is a **draft** to review, never applied.

Choose what leaves and what it does

The settings pane has two groups of toggles (all on by default):

- **Data scope** — what leaves the machine: `Inventory` · `Ports` · `SNMP health` · `Topology` · `Drift`. Turn some off for **privacy** and to cut **cost** (smaller context).
- **Capabilities** — what it may do: `Q&A` · `Diagnostics` · `Find gaps` · `Suggestions` · `Ansible draft`. E.g. turn off "Ansible draft" for a purely advisory assistant.

Clickable citations and anti-invention check

Below each answer you see the **sources it used**: the cited devices and VLANs become **clickable chips that jump to the node on the map**. A downstream check compares the IPs and MACs the model names against the real data: if it cites an address that **isn't in your data**, a warning chip ⚠ **"reference not found"** appears — a possible hallucination is *flagged*, not let through. The assistant is also fed **live facts** (the Verify result and IPAM occupancy computed in the browser and re-sanitized server-side), so it reasons over the current reality, not just the saved file.

Find gaps, suggest, draft Ansible

For "*what's missing*" the assistant uses the **gaps InfraNet already computed** (VLAN without a gateway or subnet, IPAM near full), and for "*which IP do I use*" the **next free address** of the VLAN. Ask for automation and the playbook arrives as a **draft card** with a **"DRAFT — not applied"** banner and a **Copy** button: InfraNet executes nothing — you review it and apply it yourself (manual-first).

Finally, every `Verify` row has an **"Explain"** button (robot icon): it opens the assistant and **seeds a question about that case** (absent device, IP change, undocumented host…), so the *Verify → understand → act* loop stays inside the app.

Onboarding: the «next step» and the spotlight

On first run the Assistant doesn't start from a blank page: it shows a **«next step» chip** that, based on your project state, tells you what to do *now* — empty network → `Discover`; documented but unverified →

`Verify`; VLANs without a subnet or gateway; undocumented or absent devices; IP changes. The **«Show me»** button lights a **blinking spotlight** on the *real* toolbar button: it stays on **until you click it** (and never moves the page); **«Ask»** seeds a how-to question. The chip appears **even before a model is configured**, so it can guide Discover and Verify at zero cost. Everything stays **manual-first**: it points and explains, it never applies anything.

For *«how do I X in InfraNet»* questions the assistant is grounded on the **real command surface** — the buttons with their tooltips, **derived from the UI itself** — plus a short cheat-sheet of the key flows: it cites the **real button label** (e.g. "click Discover") and never invents menus.

> **Try it**
>
> "Who's on VLAN 20?" · "Which IPs are free?" · "What's on port 5 of the switch?" · "Why is this device absent?" · "How loaded is the switch / what's the printer's toner level?". Local default: install **Ollama**, pull a model (e.g. `llama3.2:3b`), endpoint `http://localhost:11434/v1`, leave the key empty.

> **Administrators only, by choice.** The chat spends the administrator's API key and sends the project's topology to an external provider. Opening it to viewers as well would take a rate limit and a spending budget first, not the removal of the gate: it is a decision, not a setting left behind.

14 Documentation check (Drift)

Stop hoping the doc is right: the app tells you exactly where it is not.

Documentation ages: a port moved during a fault, a VLAN changed on the fly, a cable unplugged and never updated. This divergence is called **drift**. The **Verify** button queries the real devices and **compares what is documented with what is actually there**, showing you only the differences.

> **In one sentence**
>
> It stops asking you "*I hope the doc is right*" and starts telling you "*here are the 4 points where the doc does not match reality*".

> **The result is STATE, not a flash**
>
> Verify used to open an overlay window that vanished at the first click. Now the outcome **lands in the Dashboard**, in the **"Conformance"** column (chapter 15): after a Verify the view opens on that result by itself, and it **stays there as state** — it survives save/reopen, and every category of difference becomes a **navigable row** with its own actions. The same three actions as always, now *inside* the column. This chapter explains *what* Verify measures; chapter 15, *where* you read it.

The seven report categories

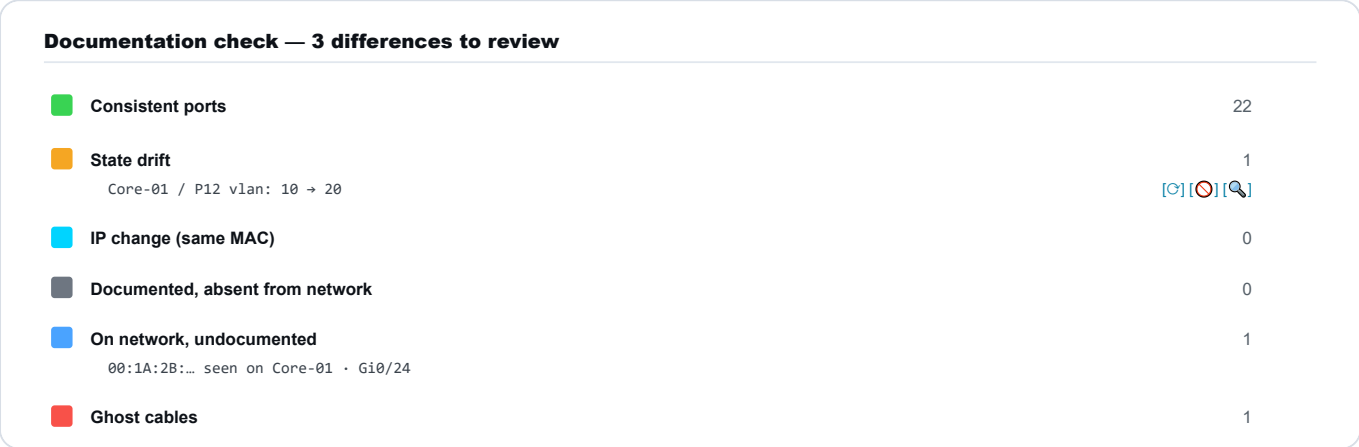


Fig. 10 — The differences, ordered by category. Each row carries its three actions — ⟳ **Update doc**, 🚫 **Ignore**, 🔍 **Investigate** — and now lives as a **navigable row** in the Dashboard's "Conformance" column (chapter 15), no longer in a separate window.

Category	Meaning
🟩 **Consistent ports**	Doc and reality match (count only).
🟧 **State drift**	Documented port active but down, or different VLAN/speed.
🟪 **Hardware identity**	The serial number or model reported over SNMP doesn't match the documented one: the device may have been **replaced** (a different firmware is informational only, not an alarm). Accept the new identity in one click, or ignore.

🟦 **IP change (same MAC)**	Device alive at a different IP (DHCP lease changed) → update the IP in one click.
⬛ **Documented, absent**	No response to any signal (SNMP, ping, ARP, TCP) *and* its subnet was reached by the check → greyed out on the map.
🟦 **On network, undocumented**	A MAC appears on the switch but is not on the map → you can "add to map".
🟥 **Ghost cables**	Documented cable with port down across several consecutive checks (threshold: 3).

LAN networks — coverage and presence per subnet

Among the rows of the **"Conformance"** column (after a Verify) is **"LAN networks"**: from the documented devices **and the DHCP leases** it derives every **/24** in the project and classifies it — 🟢 **covered** (a reachable SNMP switch → topology reconstructable by Sync), 🟠 **blocked** (the switch is there but does not answer → your action) or ⚪ **open** (no reachable SNMP switch → a scan is needed) — each with a **"Discover network"** button that opens *Discover* pre-filled with that subnet. **After a check** each /24 also shows a **presence badge** (🟢 *verified* if the sweep observed the subnet, ⚪ *not reached* if it was blind), and the **non-verifiable devices** — documented, silent, on a subnet that was not reached — are **nested under their own network** here (keeping the ignore/investigate/ explain actions), instead of a separate category: they are two readings of the *same fact*. Two distinct axes remain: *topology reconstructable?* (covered/blocked/open) vs *presence confirmed?* (the badge). Each /24 also carries a **plan-provenance** tag: **"declared /N"** if it falls inside a subnet you declared (and those come **first**), or **"undeclared"** if it's just the assumed `/24` — a reminder to put it in the plan. The *scan* still runs per /24 (a large subnet is scanned segment by segment, not in one shot).

Multi-signal presence + BYOD noise

Presence is not judged by SNMP alone: every documented IP is also queried with **ping, ARP and TCP**, so PCs, IoT devices and UPS units do not end up among the "absent". And phones/laptops of users connecting to the guest Wi-Fi are **collected in a collapsed grey row** (identified by endpoint VLAN, busy uplinks or randomised MACs), keeping the list clean. **Expanding the group, each row says why it is hidden** in plain words (Phone or tablet, Uplink to a network device, Guest VLAN), with the technical signal in a tooltip, and a «Show user/BYOD» toggle brings them back inline. Finally, **red «absent» requires proof a live host cannot suppress** — a **local ARP-miss** on the server's own segment or a **switch access port down for several syncs**: mere silence (a mute SNMP agent, filtered ICMP, a MAC aged out of the FDB) or an unreached subnet yields **grey «not verifiable»**, never a false absent, and a plain **Sync** (no sweep) never paints anything red. A device behind a router with SNMP stays **green even across subnets**, because the **router's ARP table** — or its **IPv6 Neighbor Discovery** cache — (read via SNMP) proves it alive on its VLAN, even if it is IPv6-only.

Presence colours on the floor plan (floor nodes)

After a Sync or a Verify, **floor nodes** are coloured to convey presence (**rack devices** are not: there the status is already the **SNMP LED**). 🟢 **Green** = present, from any positive signal (SNMP, a MAC in an FDB, an active lease, ARP — including a **router's ARP** or its **IPv6 Neighbor Discovery** cache seeing it alive on another VLAN: green **across subnets**, without a ping, even if IPv6-only). 🔴 **Red** = *proven* absent: only a **local ARP-miss** on the server's segment or a **switch access port down for several syncs** — signals a live host cannot fake. ⚪ **Grey** = not verifiable: ambiguous silence (a mute SNMP agent, filtered ICMP, an aged FDB, an unreached subnet) — the

honest "don't know". With the **«Released lease = likely disconnected»** option on (in the DHCP lease import panel), a grey device whose DHCP lease is *released* is annotated "likely disconnected" — it still stays grey, never red (a hint, not a proof). A **Sync** never paints red (no active probe). Documented devices **without a MAC** (IP only) are judged too, per-device.

Management VLAN: the opposite of a guest VLAN

You can mark a VLAN as a **"management" VLAN**. It's the opposite of a guest VLAN: an *unknown* device showing up there is never hidden as a phone/BYOD, but treated as **infrastructure** and flagged with a red **"On management VLAN"** security badge — because an intruder on the management network is exactly what you want to spot at once. Adopt already proposes it as a network device.

Automatic IP renewal (opt-in)

You can enable automatic IP renewal from DHCP: the device's identity is its **MAC**, the IP is just an attribute that rotates. IPs set manually are *never* touched, and every renewal is logged in History.

DHCP leases as a source (cross-VLAN)

Behind a firewall that routes between VLANs, the local ARP table is blind: an IP change on another VLAN is missed. **DHCP leases** instead cover *every* VLAN. From `Automations` → `DHCP` section → `DHCP leases` → `Load` you **paste or load** the lease table (format auto-detected: ISC dhcpd, dnsmasq, Kea CSV, generic CSV — pfSense / OPNsense / MikroTik / Synology / Windows). Leases feed the same documentation-check engine: **cross-VLAN IP change**, per-MAC presence and **undocumented devices** with their hostname. Multiple servers **accumulate as persisted sources** (merged per MAC, newest lease wins); per source: refresh, clear, remove, "verify now".

An identity map, not a liveness probe

A lease table says *who has which IP*, not *who is powered on*: a documented device **missing** from the leases lands in **Not verifiable**, never among the absent (absence from a DHCP list does not prove the device is off — it may have a static IP). **Expired** leases are ignored, and a **manually-pinned** IP is never silently overwritten: the row is flagged "Manually pinned IP" and asks for confirmation. Loading from file/paste is built-in; **live** pull via vendor APIs (FortiGate / PAN-OS / OPNsense / MikroTik) is an optional driver pack.

The result does not wait for Save

Who is there and who is not is **stored the moment it is measured**, without you pressing Save: that reading is not an edit you made to the document, it is a **measurement of the network**. Reopen the project and devices that are switched off are still red — even with autosave off, and even when the check was run by the **automatic monitor** while you were away.

The document stays yours

> *Your* unsaved edits stay unsaved, and the project file is not rewritten behind your back: presence lives beside the history, not inside the document. If a fresher measurement already exists, that one wins — old data never drags newer data backwards.

15 Dashboard (summary view)

Three questions, one screen: what's missing, what no longer matches, how much room is left.

The **Dashboard** is a **read-only** view — a view switch like Topology, that **never touches the document**. It answers three questions in three columns: **LAN** (does it describe everything?), **Conformance** (does it still match reality?), **Expansion** (how much can I still grow?). Each tile carries its **own number** and a **plain-word verdict**: a "25" on its own doesn't say whether it's fine — the word does.

In one line

It's not a metrics dashboard: it's the answer to "is my document complete, current, and with room to grow?", with — next to every number — **where it came from**.

Who counts in "Verifiable over SNMP"

One thing decides it: the driver in **Properties** → **Integration**. An SNMP driver set = "I poll it, it must answer" → it enters the count. No driver = "I don't poll it" → it leaves the count and joins the **"not verifiable"** list. This holds for *any type*: a printer speaks Printer-MIB just as a switch speaks IF-MIB, so what decides is not what the device is but whether you poll it.

Two counters, and the names

The tile carries **two numbers** — two different populations that never add up: how many **answer** among those you poll ("*12 of 13*") and, next to it in amber, how many you **don't poll at all** ("*20 not verifiable*").

With several racks, "2 not responding" forces you to leaf through the pages to find out *which*. So when something is wrong the verdict writes the **names**, right there, without a click — at most three, then "+N".

- **No answer** (red) — it is configured but the last read *failed*: "*no answer: OfficeJet8715*". That is a measurement gone wrong, and no green is painted over it. Fix the credentials, power the device on, or stop polling it.
- **Green** — once everything you poll answers. The second counter keeps saying how many stay out, and one click on the row opens the full **list** of the not-verifiable ones, each with its name and address: that is your list of in-person checks. With *zero* accesses configured the row isn't green at all: it is dashed — polling nothing is not a good result.

The list covers **every** device the panel offers SNMP Integration for — the ones that could have an access: not just switches and routers but printers, NAS, cameras, UPSs. Not being verifiable over SNMP does not mean being unchecked: **Verify** still checks them by *presence* (a MAC in an FDB table, ARP, ping). It means that to know how they are *inside*, you have to go and look.

What's new: Verify lives here

The middle column, **"Conformance"**, has become **Verify's home**. When you launch a **Verify** (from the toolbar), the Dashboard opens on this column by itself and drops the result there: no longer a window that disappears, but **state** that stays — how many differences are left to decide, when it was done, and **one row per kind of**

difference that opens in place with its actions. The “Conformance” column tells the *health* of the documentation over time.

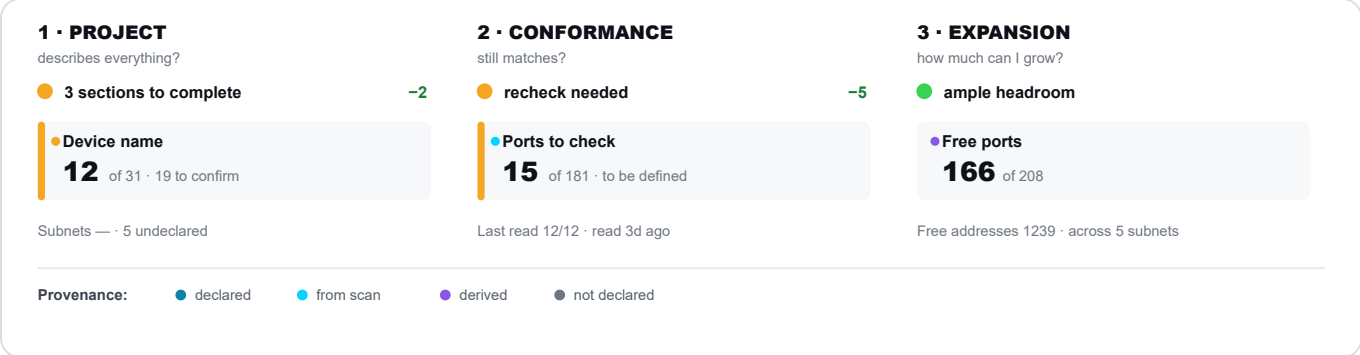


Fig. 11 — The three columns on *Rete+Lab*. Each opens with a **health-dot verdict** and, when a previous read exists, the **since-last-read delta** (−2, −5). The most urgent tile carries a **coloured left bar**; every number declares its **provenance**. After a Verify, the “Conformance” column fills with the result (Fig. 12).

The “Conformance” column: where Verify lands

After a **Verify**, the middle column no longer shows just “when” the last read was: it fills with **the outcome**. At the top, the **“when” strip** carries two dates — the last *Sync* and the last *Verify* — because they are the date everything below is valid at. Beneath, **one row per kind of difference** found: ports that changed state, devices on the network that aren't documented, ghost cables, hardware identity, changed addresses. Each row opens *in place* and shows the concrete cases with the **three actions** as always — **Update doc**, **Ignore**, **Investigate** — without ever leaving the Dashboard.

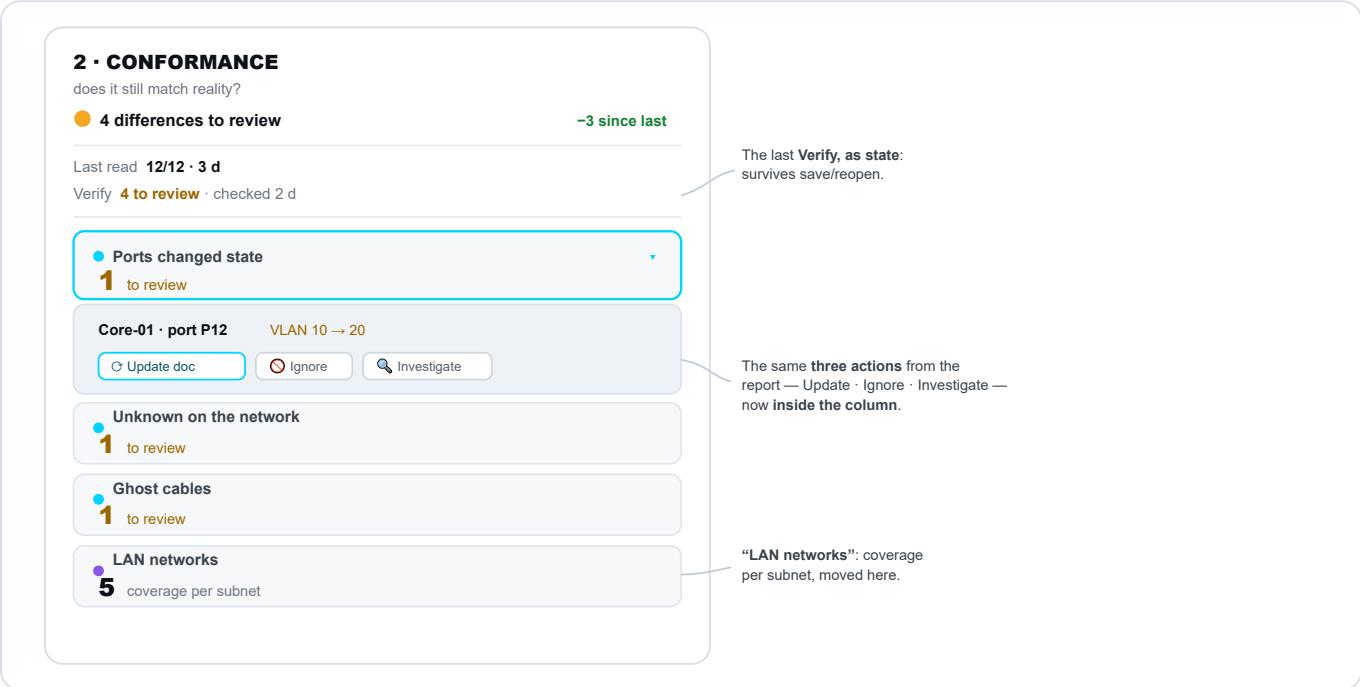


Fig. 12 — The **“Conformance”** column after a Verify. The top strip carries the two dates (**last read** and **last Verify**); below, **one row per kind of difference**. The open row shows the concrete case with the three actions — **Update doc**, **Ignore**, **Investigate** — **inside** the column. The coloured dot declares the **provenance** (measured · derived).

Where every number comes from

The Dashboard's hallmark is that it **never presents an estimate as a fact**. Under each number a dot says where it came from, and a **missing datum is shown**: it renders as a **dashed "not declared" tile**, not a zero — because *a zero is an assertion, a dash is a question*.

Provenance	Meaning
Declared	You wrote it in the document.
From scan	Read from the device at the last SNMP sync/check.
Derived	Worked out from other signals (e.g. free addresses, with the `/24` assumed).
Not declared	The value isn't there: a dashed tile, never a zero.

Subnets go on the DECLARED prefix

The **addressing plan is law** (see chapter 12). If you declare a subnet in the VLAN panel — a `/16` as much as a `/28` — the Dashboard uses it **at its real prefix**, not a convenient `/24`. So **"Free addresses"** (Expansion) counts the true capacity (a `/16` is ~65,000 addresses, not 254) and **"Subnets"** (LAN) lists the **declared ones first**. Networks you **use but haven't declared** don't vanish: they show up tagged **"undeclared"** (with the assumed `/24` as a fallback) and the verdict becomes **"N to declare"** — a nudge to complete the plan. And if a declared subnet is **smaller** than what you've put in it, the devices that fall **outside** surface there as "undeclared", instead of being hidden or double-counted.

The glance: one verdict per column

At the top of each column, the **answer** to its question: a **health dot** (🟢 fine · 🟡 minor gaps · 🔴 serious problem) with a sober phrase. **Red is reserved for "flying blind"** — a network *never read*: a project read recently but with some discrepancy stays **amber**, not alarmist. Beside it, when a previous read exists, a **since-last-read delta** (**−N** problems fewer · **+N** more).

Where to start

Within each column, the **most urgent** tile takes a **coloured left bar** — amber or red by severity — so the eye knows where to begin. If the column is fine, no bar: putting the accent on "166 free ports" would say something false.

Looking never dirties the document — with one deliberate exception

The Dashboard is **read-only** for *navigation*: opening it, clicking inside, going back **never marks the project as modified**. The chosen view and the "since last read" comparison live in the **browser** (localStorage), never in the document. The **only exception** are the **three actions** in the "Conformance" drill-down (*Update doc* · *Ignore* · *Investigate*): those **do** touch the document and mark it modified — because *deciding* a difference is, in every sense, an edit, and it is exactly what you want to do from here.

Every row drills down in place

> A click on a row **lists what's behind it**, *inside the column* — the overview never disappears. **Derived cables** show both ends resolved to names; **LLDP/CDP neighbours** resolve a bare chassis-MAC to the project device's name; **free addresses** give free *of usable* per subnet — on the **declared prefix** when there is one, each row tagged "declared"/"undeclared"; **free ports** count **switch** capacity (the port where a new device actually attaches) — patch panels, appliances (NVR, KVM, console server, NAS, firewall…) and **wall outlets** have free ports but are not network fabric, so they are counted **apart** (a "+N non-switch" badge beside the number, marked "non-switch" in the drill-down); the list splits into two tabs, **In-rack** and **Outside-rack** — the latter gathers the free wall outlets (free of total, per device). And in **"Conformance"**, the difference rows open the **cases to decide** directly, each with its three actions: it is the one place in the Dashboard you *act* from, not just look.

> **The detailed reports live here**
>
> Where a tile already carries the verdict, its **full report** opens from the drill-down: the **free-ports table** (with **CSV** export) from the *Headroom* pane, the **L3 → gateway map** — also exportable — from the *Gateway* tile. These were items in a separate "Reports" menu in the toolbar: now the Dashboard is the way in, one datum and one place. The toolbar keeps only **Change history**, which is a log over time rather than a state snapshot.

The six lenses: from the summary to recoverability, security and health

At the top of the Dashboard a **lens switch** — `Summary` · `Recoverability` · `Security` · `Health` — changes *what* the view shows you, without ever touching the document (the choice lives in the browser). **Summary** is the three columns above; the other three are **full-width**, opt-in lenses that answer questions other than "is the document complete?".

④ Recoverability (DR) — "if it falls tonight, can you bring it back up?"

The question that matters when something really breaks. The lens counts the **managed devices** and gives an **"X of Y recoverable"** verdict, where *recoverable* means having all three pieces needed to rebuild it:

- **A fresh config backup** — the pointer to *where* the backup lives (never the file), updated within the last **30 days**.
- **Known hardware identity** — you know *what to re-buy*: make/model (declared or read over SNMP) *or* the serial; a missing **firmware** is a light advisory, not a blocker. If the *declared* serial does not match the *measured* one, the device is flagged "replaced?" and does **not** count as recoverable: that is a doubt to settle, not an identity to rely on.
- **Location** — the rack it goes back into.

Alongside, three *advisory* signals that never lower the verdict: **presence** (seen recently vs. stale), **redundancy** (devices with a declared **HA** twin — your single points of failure surface by difference) and **lifecycle**. Every row opens and lists the devices missing that piece — with the IP right next to the name.

> **Lifecycle — "when it dies, can you still replace it?"**
>
> In every device's **Properties** panel, next to serial and firmware, there are two dates: **Warranty until** and **End of life (EOL)**. No device reports them over SNMP — there is no OID telling you when a contract expires — so they are **declared and nothing else**: an empty field means "I don't know", never "unlimited". The row sums them up

> worst-first: **end-of-life** (past EOL: you can't buy it again), **out of warranty**, **expiring** within **90 days**. The denominator is *whoever declares a date*, not the fleet: "12 of 12 covered" while counting the twenty that declare nothing would be a green bought with ignorance — which is why the verdict also says how many are left without dates. It is *advisory*: an end-of-life device with a fresh backup is still **recoverable**, because EOL says you can't *re-buy* it, not that you can't bring it back up.

⑤ Security & Services — "how exposed is management?"

How open the network's *service door* is. Measured or declared signals only, never invented:

- **Encrypted SNMP access** — how many devices speak **v3** (authenticated) and how many still **v1/v2c** (cleartext community); the drill-down lists the weak ones.
- **Default community** — how many use a *guessable* community (public/private). The **raw value** *never* leaves the engine; for an **administrator** the drill-down shows *which* known default (public/private/empty) each device uses — the name of the flaw, not a secret — while a *custom* community is never revealed.
- **Management VLAN** — how many VLANs you marked as "management" (segmentation of the management plane); the drill-down lists them **by name**, and there can be more than one.
- **Wireless VLAN coherence** — how many **SSIDs** advertise a VLAN their access point's *uplink* does not carry (declared on the Wi-Fi but absent from the *trunk* upstream): a segmentation that does not hold. The drill-down lists the out-of-place SSIDs, with the AP that emits them; **zero = coherent**. It used to be a report of its own in the "Reports" menu; now it is a verdict.

> **The honest verdict**
>
> **Red** if a guessable community is reachable (an open door); **amber** for cleartext SNMP, no management VLAN, or a wireless SSID outside the trunk; **green** when management is hardened. And **grey** until you have configured *even one* SNMP access: zero exposure measurements does not mean zero exposure — the app cannot know whether those devices do not speak SNMP or whether nobody has written the access down yet. A green dot there would be reassurance bought with ignorance.

⑥ Health — "what problems are there *right now*?"

The only lens that speaks about the **present**: not the document, but how the devices are doing at this moment. It adds no polling — it **composes the telemetry the devices already returned** during Verify:

- **Resources (RAM · disks)** — memory in use and full volumes, read over HOST-RESOURCES.
- **Power backup (UPS)** — on battery, remaining runtime, charge, output load.
- **Consumables** — toner and ink running out (Printer-MIB).
- **CPU load** — the busiest among those that reported it. It is a *number*, not a verdict: a threshold on an instantaneous reading would be invented, and a spike between two polls is not a fault.

Each row's **denominator is whoever actually answered** with that telemetry: a NAS that doesn't speak Printer-MIB is not "a printer with no alerts", it is simply not in the count — and the row stays **dashed** instead of showing a zero.

> **Yesterday's health is not today's health**

> With no reading at all the verdict is **grey** (“I don't know”), never green. And a green one **degrades to amber** once the **oldest** reading passes **6 hours**: the picture is no fresher than its stalest piece — one device re-read just now does not refresh the others. Run **Verify** again to bring it back to the present.

> **“What I'm not looking at” — the declared perimeter**
>
> At the foot of the Dashboard, under every lens, a quiet line names the network dimensions this summary does **not** judge, so the absence of an alarm is never read as “all clear” (each chip's full text sits in its *tooltip*): WAN/circuits, L3 routing, spanning-tree, firewall/ACL, AAA·syslog·NTP, backup-restore proof, temperature and interface errors, trunk symmetry, port access control (802.1X/NAC). The list is **live**: when a dimension gets modelled it drops off on its own — which just happened to *UPS power* and *device health* (now with a lens of their own) and to *warranty/EOL* (the “Lifecycle” row).

> **A verdict has an age too**
>
> After **seven days** with no Sync or Verify, a green verdict in **Conformance** and **Expansion** turns amber and reads *“read N days ago — re-verify”*: those are the two columns that rest on a measurement, and “ample headroom” after weeks with no contact is a memory, not a fact.
> The gate deliberately spares **LAN** — which asks whether the *document* describes everything, and a document does not age — and **Recoverability**, which already has its own clock (the 30 days on the backup): two alarms on one verdict would read as two different facts.

In “Conformance”: IPAM conflicts and VLAN names from the network

The centre column carries two checks a real IPAM makes and that used to stay in the panels:

- **IPAM conflicts** — the **same IP** on two documented devices (VM IPs included) or two **VLANs whose subnets overlap**. Zero conflicts = green; otherwise the drill-down lists the two devices and the shared address, or the overlapping subnets.
- **VLAN names** — the names *read over SNMP* compared against what you declared. The Dashboard fills the gaps (a VLAN in use with no declared name, but the network knows one) and flags the **conflicts** (your name ≠ the network's, or two switches disagreeing). The **declared name is never overwritten**: you decide, from the VLAN panel.

16 REST API and automation (Ansible)

Let other tools read your documentation — read-only, with a token.

InfraNet can become a **programmable source of truth**: dashboards, wikis and automation tools read the network inventory straight from the app through a **read-only REST API**. The principle stays *manual-first* — you write the truth in the interface, machines **consume** it without changing it.

Access tokens

An external tool doesn't have your browser session: it authenticates with a **token**. You create one from the user menu → `Users and access` → `API tokens` tab:

- pick a **label** (e.g. `ansible`) and press `Generate token`;
- the token is shown **only once**: copy it right away and keep it safe;
- the list keeps only the **prefix** and the last-used date; you can **revoke** it anytime.

> **Admins only, read-only**
>
> Token management is restricted to admin accounts, and tokens grant **read-only** access: they cannot change your data.

What the API exposes

Responses are **sanitised**: you get each device's type, IP, MAC, VLAN, rack and brand, but **never** secrets such as SNMP communities. The VLAN is derived automatically from the IP address.

Endpoint	What it returns
`/api/v1/projects`	the list of projects
`/api/v1/projects/{id}`	the full inventory: VLANs, racks, devices
`/api/v1/projects/{id}/devices`	just the device list
`/api/v1/projects/{id}/ansible-inventory`	the dynamic inventory for Ansible
`/api/v1/openapi.json`	the full API description (OpenAPI)

```
# project 8 inventory, passing the token in the header
curl -H "Authorization: Bearer inp_…" http://<host-ip>:8421/api/v1/projects/8
```

Dynamic inventory for Ansible

This is the main use case: InfraNet is the **source of truth**, Ansible the **executor**. The `integrations/ansible/` folder ships a ready-made script (`infranet_inventory.py`, Python standard library only) that turns a project into a **dynamic inventory**: every device with an IP becomes a host, grouped automatically by `type_*`, `vlan_*`, `rack_*`, `brand_*` and `backup_missing`.

```
# three variables and you're set
export INFRANET_URL=http://<host-ip>:8421
export INFRANET_TOKEN=inp_…
export INFRANET_PROJECT=8

# view the groups, then target a group
ansible-inventory -i infranet_inventory.py --graph
ansible type_switch -i infranet_inventory.py -m ping
```

No hand-kept inventory

When you update the documentation in InfraNet, the Ansible inventory follows: one source to keep current, not two.

Configuration backup — the pointer, not the file

For each device you can record, in the Properties panel → `Configuration backup`, **where** its configuration backup lives (a path or a repo) and when the last one was taken: InfraNet notes the *where*, **never the configuration file** — which holds secrets. The Ansible inventory then carries, per host, the **network OS** (so the right module is picked) and that **pointer**, and gathers into the `backup_missing` group the devices **without** a recorded backup: a backup playbook finds them on its own — and the **AI assistant** can draft that playbook for you. The path accepts no credentials.

17 Change history

Who changed what, and when. An append-only log that survives Undo.

On a documented network, multiple people work over time. Sooner or later the question comes: **"who changed this, and when?"**. **History** is the chronological log of project changes — an *append-only* diary: entries accumulate, they are never deleted.

> **In one sentence**
>
> It turns the map from a "snapshot of today" into a "film of how we got here".

What is recorded

Only **structural events**, not every minor tweak: device added/removed/renamed, cable created/removed, port VLAN changed, Sync and Verify run (with outcome), documentation aligned, project renamed. Every entry carries **date and time, user, action, object and detail**.

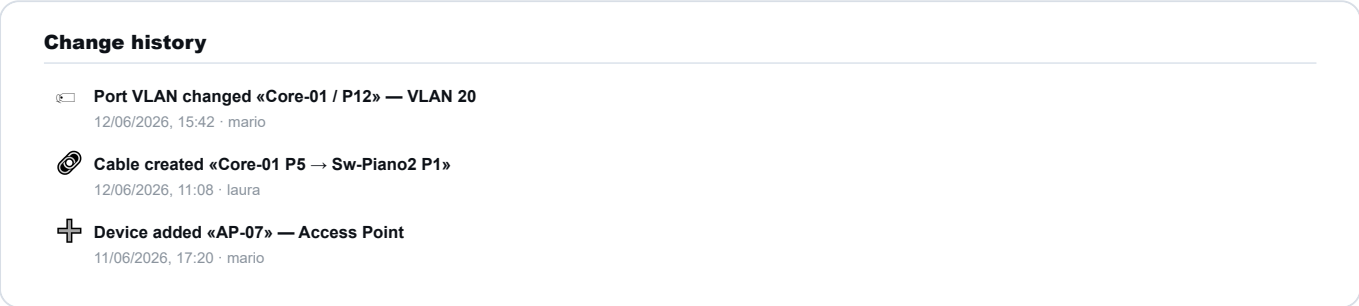


Fig. 13 — The log, most recent at the top. Filter by device/user/action and **export as CSV** for handovers and audits.

- **Cannot be undone.** Unlike everything else, History survives Undo/Redo: a log that can be erased would not be reliable.
- **Export as CSV** to attach documentary proof (handovers, audits, proof of work for MSPs).
- Only meaningful if **each person uses their own credentials**: with a single "admin" account, the "who" column loses its value.

Verifications over time and restore points

The same window now has **three tabs**. **Changes** is the log above. **Verifications** is the *timeline*: every Verify (manual or scheduled) leaves a lightweight row with date and divergences, so you can read the network's health over time. **Restore** keeps **full snapshots** of the state: you can *create a point* whenever you like and **restore** it later — before doing so the app automatically saves a safety point. A snapshot is also taken on Save and before risky operations (import, bulk adopt). The whole history lives **outside the project file**, so the data doesn't bloat, behind an interface already ready for a database.

18 Export, labels and Dossier

Getting documentation out of the app: SVG floor plan, cable labels, handover dossier.

Floor plan and backup

- **SVG** — the printable floor plan, with VLAN-coloured cables and device icons. If a VLAN filter is active, the SVG exports only that VLAN.
- **JSON** — the project in a **versioned format** (it carries its schema number), for moving it between installations. It is **secret-free**: SNMP communities/credentials and backup references are **stripped** on export — no longer a backup of passwords, but of the network. On import InfraNet accepts both the new format and older saved files.
- **CSV** — bulk import of tens of devices from an Excel/CMDB spreadsheet.

> **SVG = recommended format for the floor plan**
>
> The SVG export keeps the floor plan **vector**: you can enlarge it and print it at any size — even on a plotter or large format — **without losing sharpness**, unlike a raster image (PNG/JPG) that pixelates. It is the preferred format for archiving and delivering the map.

Export the rack to draw.io

From the **Import/Export → Export to draw.io** menu you get a `.drawio` file (the diagrams.net format) with **one page per rack**. It is not a screenshot: the frame, the devices and the ports are **native, editable shapes** inside draw.io's own rack container (each device snaps to its U slot). Open it in diagrams.net and move a server, rename a port, add a note — it all stays editable, with no "pasted-in" images.

- The **device names** sit **outside the rack**, to the side, so the device face is left entirely for the interfaces.
- The **SNMP status stripe** is not exported: it is a *live* indicator, whereas the `.drawio` file is a *static snapshot*.
- If you gave a device a **skin** (a drawn faceplate), its artwork is carried over as a vector image with the real ports overlaid.

> **One cable layer per VLAN, with a table**
>
> The export includes the **intra-rack cables** as native connectors, split across **separate layers — one per VLAN**, each **named with the VLAN's name**, cables **coloured by VLAN** and routed in dedicated lanes (with a stagger) so they don't overlap. In draw.io open **Edit → Layers** and turn on the VLAN you care about: alongside its cables a **table** appears listing the connections — one cable per row (*Device/port → Device/port*).

> **Isolate a cable: click the row to highlight it**
>
> In diagrams.net's **View** (preview/lightbox) mode, **clicking a table row highlights that cable**: the line becomes thick and *stays* highlighted (one at a time) and the view scrolls onto it. **Clicking the table header** clears the highlight. (In the editor a click opens the connector's link instead; the highlight works in View mode.)

A4 format, or A3 if it doesn't fit

Each page is exported as **A4 portrait**; if the content doesn't fit — a very tall rack or a VLAN with a great many cables — the page switches automatically to **A3**.

What about Visio?

Visio does not open `.drawio` files directly. If you need it, open the file in diagrams.net and use **File → Export as → VSDX**: you get a file Visio opens, with a "flattened" result (draw.io's native rack shapes have no native Visio equivalent).

Printing cable labels

From the **Import/Export → Export labels** menu you generate cable labels ready to print, on standard market media:

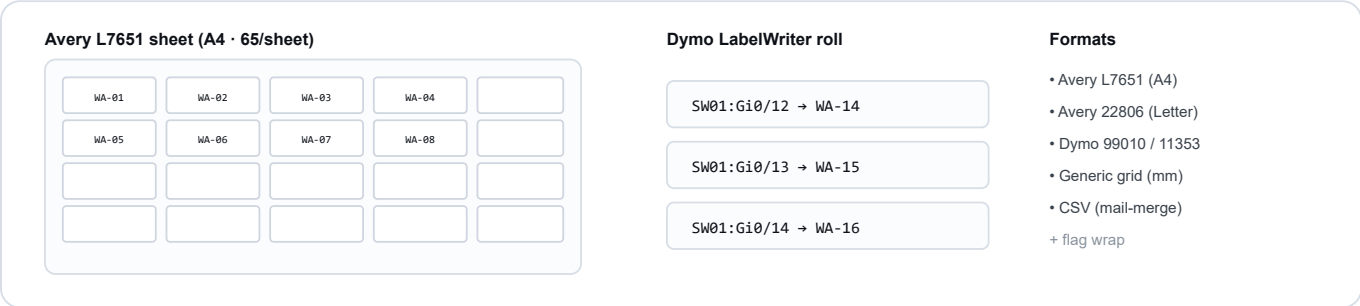


Fig. 14 — Labels on **Avery** sheets (L7651 A4, 22806 Letter), **Dymo** rolls (99010, 11353), generic grid configurable in mm, or CSV for mail-merge. Preview is live.

- Choose the **fields** to print (by default only the label ID) and the **flag wrap** option, which repeats the ID in both halves → readable from either side when wrapped around the cable.
- Geometries are nominal: do a **test print** and align it against the label sheet to check alignment; any offset can be corrected with the "generic grid".

Handover dossier

A single PDF, with one click

The **Handover dossier** collects everything the recipient needs: cover page with key figures, floor plan, **Dashboard**, rack fronts, cable inventory, as-built path trace, port assignment, VLAN/IPAM, **virtual machines**, **power (PDUs)**, asset register and change history. For an MSP it is a *billable deliverable*; for a company it is *insurance against staff turnover*. Generated from the Import/Export menu.

Power: one restore sheet per PDU

The **Power** chapter opens with a totals line (units · outlets · active · loaded · free · faulty), then gives **each PDU a sheet** with everything needed to put it back: identity (brand, model, serial, firmware, asset tag, warranty), position (rack, unit, height, mounting), electrical rating (type, phases, rated current), management (mode, ports, IP and MAC address), the **backup pointer**, what it is **powered from**, and the **outlets and loads** list — for every outlet its state, the device it powers, that device's power port, and whether the link was *imported* or set *by hand*.

A sheet never splits across two pages: tear off the page and you have that PDU whole. An undeclared field prints a dash, *never* a zero; and a PDU that declares an outlet count without listing the outlets says so, instead of passing a row of zeros off as a measurement.

Notes sit next to their device

The **note** you wrote on a device no longer ends up in a chapter of its own: it is printed **under its own row** in the **Asset register**, and the register subtitle says how many devices carry one. A separate chapter forced the reader to match names back to devices before the note meant anything; this way identity, location and the engineer's recommendation are read together. Notes written on *structural cabling* (wall outlets, electrical panels) are not printed: the asset register lists IT assets, not the building's wiring.

The Dashboard up front

Right after the floor plan, one page with the **three questions** and their verdicts — the same ones you see on screen, with the same **provenance dot** next to every number and its legend. It is the *executive summary*: whoever opens the dossier reads the judgement first, and only then the data backing it in the pages that follow. It lists no devices (that's the **Asset register**): it fits on **one page**, so it actually gets read. At the bottom, the **"What I'm not looking at"** line travels with the dossier — putting in writing what was *not* assessed protects whoever hands it over as much as it is honest towards whoever receives it. It can be switched off in the export options like any other section.

Virtual machines in the dossier

VMs get a **chapter of their own**: one row per machine, grouped by host, with the role or service provided, state, VLAN, IP address, allocated resources, owner and criticality. On the cover they have **their own counter**, next to devices, cables and VLANs: they are *never* added to the device count, because a host with ten VMs is still one installed appliance and inflating that figure would mislead the reader. Whatever you left blank is printed as «-»: the dossier invents nothing.

The "Recoverability (DR)" section — the runbook to rebuild from

An optional page, **one row per managed device**, built for the day of the disaster: **where** each device's config backup lives (the pointer and method, *never* the file or the credentials), the **date and time** of the last backup, the **serial and firmware** to procure and reflash, the **lifecycle** (the declared warranty and end-of-life dates, summed up by the worst state: *EOL* · *out of warranty* · *expiring* — "can you re-buy it?" comes right after "what do you re-buy?") and the **rack** it goes back into — under a header verdict **"X/Y recoverable · N without a backup · N without a location · N with an unresolved identity · N end-of-life"** (the last four appear only when greater than zero). The thresholds are the *same* as lens ④ (backup fresh within 30 days, expiring within 90): the dossier you hand over cannot say something different from the app. Kept *off-site*, this page lets you rebuild the LAN even with InfraNet down. Same secret-free allowlist as the asset register: no community, no password, never the configuration file.

For a partial PDF (floor plan only, rack only…) there is the classic **Export PDF** with checkboxes to choose individual sections. Tip: run a *Sync* just before exporting, so ports, VLANs and status in the dossier are up to date.

19 DCIM / IPAM import (NetBox)

Build a new InfraNet project from an existing NetBox — read-only, and with no surprises.

If your network is already described in **NetBox** — the most widely used open-source DCIM/IPAM — you don't have to redraw it from scratch: InfraNet **imports** it and creates a ready-made project with racks, devices, VLANs and cabling. The import is **free** and **read-only**: it reads from NetBox, and never writes back.

> **Where to find it**
>
> Menu **Import/Export** → **DCIM/IPAM sync**. Write-back to NetBox (export) is a **paid module**: in the free build the "Export" tab stays locked.

1 · Connection

Enter your NetBox **base URL** and an **API token**, then press **Test connection**: the button turns **green** if it answers, **red** if it doesn't (with the reason). The token is kept **server-side only** (permissions 0o600), never comes back to the browser and never lands in the saved project — the same handling as the SNMP community and the AI assistant key.

2 · Import in three steps

A small wizard walks you from choosing what to import to the new project:

Step	What you choose
① **Scope**	The **site** to import (with counts next to it). You import **one site at a time**.
② **Entities**	What to bring in: **Devices + ports + cables**, **IPAM** (VLANs and prefixes), **Racks**.
③ **Preview**	The **estimate** of what you will get, the **list of decisions** (see below) and per-row deselect → **Create project**.

On confirm a **progress** screen appears, then a **result** with the counts and an **Open project** button.

> **One site = one project**
>
> The site names the project, racks keep their NetBox names, and a device's *Location* becomes a note (InfraNet has no multi-floor model). Import one site at a time so rack names stay unambiguous.

3 · What it rebuilds, and how

- **Racks on the floor plan** — each imported rack is placed on a non-overlapping grid and shows as a clickable floor icon; the first rack opens already populated in the Rack view.

- **Front/rear split** — a NetBox cabinet with devices on *both* faces becomes **two** InfraNet racks (the rear as "*name* · retro"), each device on its own side; cables crossing the two faces become links between the two racks.
- **Patch-panel cabling** — front/rear-port terminations are rebuilt as a **native pass-through chain** (switch → panel-A → panel-B → server), with no synthetic segments. The power **cable** and WAN circuits don't become topology links: a PDU's connection lives in the *outlet* (see below), not in a drawn cable.
- **PDUs and power** — if the site has PDUs, InfraNet imports their **outlets** (up to 48) with **status** and the **powered device**, the power **inlets** and the **management** ports. How they read and edit is in the **"Hardware catalog, PDU and combo ports"** chapter.
- **Stacks** — a NetBox *virtual chassis* becomes a **stack** again: the chassis name is the stack name, the position the member number, the master the master. Members stay **separate devices** — they are separate boxes in separate rack units — but they now know they belong to the same stack, so a port on member 2 no longer looks like an orphan.
- **Catalogue** — the NetBox model is matched against the built-in catalogue (native port count and front panel); if it isn't there, the imported interface count is used.
- **Platform and description** — the *platform* declared in NetBox picks the network OS for the **Ansible inventory** (it used to be inferred from the brand); the device *description* lands in the **notes**, next to site and location.

4 · The preview is a list of decisions

The third step does not list warnings: it lists **decisions**. At the top sits the **estimate** — "I will import 72 devices · 50 cables · 7 VLANs · 24 racks · 4 stacks" — then the rows, ordered by what they cost you:

Section	What it holds
What will not come in	What stays out *because of how the model is built*: console ports and power feeds on anything that is not a PDU, power cables and WAN circuits. With the **count** and the **names** of the devices, not just a number.
To decide	The real choices, with the alternative and its consequence, and the **default** already applied. One row is *one decision*, never one device: thirteen switches with the same case are a single row.
No choice needed	Declared limits that lose nothing: VLANs that merge, models outside the catalogue.

What the DCIM declares is on screen. At the bottom of the **Properties** panel of an imported device sits the **Declared by the DCIM** block: owner (*tenant*), declared status, role, platform. It is **read-only** on purpose — that is NetBox speaking, the fields above and the notes are yours — and it shows **only** what the DCIM actually declared: a printed blank would read as data.

Touch nothing and you get what you would have got before: the defaults **are** the historic behaviour. Changing a decision and pressing **Recalculate** is instant, because the NetBox read is reused instead of repeated. That is exactly why the panel says **what time** it was read, with **Read NetBox again** next to it to go and fetch it for real.

Why names, not just numbers

A preview that says "58 cables not imported" leaves you wondering *which*. Every row opens the list of the devices involved: the decision is yours, but on data with a name on it.

Manual-first and non-destructive

The import always creates a **new project**: it never touches or overwrites an existing one. It writes nothing to NetBox — write-back is a separate operation, in the paid module, and even then additive only (create or update by natural key, **never delete**).

20 Hardware catalog, PDU and combo ports

Give every device its real ports — power outlets included — without inventing them.

A well-drawn device has **the ports it actually has**: not one more, not one fewer. InfraNet starts from a **NetBox-compatible hardware catalog** to recognise models, treats **PDUs** (the rack's power strips) as first-class citizens, and never double-counts **combo ports**. Everything stays **manual-first**: what you type by hand always wins.

1 · Apply a model (NetBox-compatible catalog)

In the **Properties** → **Ports** panel of a switch, router, firewall or access point you can pick the **real model** from a catalog of thousands of devices (NetBox's CC0 library — the same one the DCIM import from the previous chapter uses). The device then inherits its exact ports — **copper, SFP/SFP+, QSFP, management** — in the right number and order, instead of a generic count.

> **Vendor-neutral**
>
> The catalog describes ports in a **vendor-independent** language: applying a model does not invent or reorder ports, and adds no brand-specific rules. The catalog is refreshed from the NetBox library with one command (`npm run update-device-types`) without touching your projects.

2 · PDU — the rack's power outlets

A **PDU** (Power Distribution Unit) is the rack's power strip. InfraNet draws it with its **outlets** — up to **48** — inside the rack frame, as a row of cells that shrink as density grows; the frame grows with the PDU's height (1U, 2U and beyond).

In the **Properties** → **Power** panel — **populated by the NetBox import** when there is one, or filled in by hand — you say, outlet by outlet, **which device** it powers and on **which inlet**. Outlets are not network ports: you don't run a data cable to them and they don't count towards spare ports.

Outlet state	Meaning
Active	Outlet with a device connected and powered.
Inactive	Free or switched-off outlet — the default state of an undocumented outlet.
Faulty	Outlet flagged as failed.

> **Your word wins**
>
> If you import the PDU from NetBox, the imported connections stay the **source**; your manual edits (device, inlet, state) are stored as **separate overrides** and can be **reset** back to the imported value without losing it. The PDU also keeps its auxiliary ports — **Ethernet management, serial, sensor, USB, expansion** — separate: only the Ethernet management port acts as a network port.

3 · Combo ports

Many switches have **combo** ports: one physical position shared between a copper port and an SFP slot, where you use **one at a time**. InfraNet models them as **one** position, not two — so the port count stays honest and a spare port is never counted twice.

> **Why it matters**
>
> On a device with many combos, counting them as separate ports would inflate the real capacity. Treating them as shared slots keeps the drawing, the spare-port count and the topology aligned.

21 Bilingual IT / EN

Interface in Italian or English, with networking terms always kept in their original form.

The interface is **bilingual Italian/English**. The selector is in the user menu; the choice is remembered between sessions. You switch between languages without reloading and without losing your work.

> **The technical glossary is not translated**
>
> Networking terms — **VLAN, SNMP, LLDP/CDP, SFP, trunk, uplink** — and vendor names remain unchanged in both languages: translating them would only cause confusion. What is translated is the interface (menus, panels, messages), not the language of the trade.

Bilingual coverage is verified automatically: a check flags any string translated on only one side, so both languages stay aligned as the app grows.

Support the project

InfraNet Pro is **free and open source**. If it saves you time, you can buy the development a coffee: every contribution helps fund new features, fixes and support.

> ☕ **Buy me a coffee**
>
> Support page: **ko-fi.com/infranetpro**